Journal of Building Engineering 104 (2025) 112222

Contents lists available at ScienceDirect

Journal of Building Engineering

journal homepage: www.elsevier.com/locate/jobe

Optimized design of RC moment frames with machine learning methods

Fateme Heydari [a], Masih Andalib [a], Siamak Epackachi [a,*], Reza Rafiee-Dehkharghani [b]

[a] Department of Civil, Water, and Environmental Engineering, Amirkabir University of Technology, Tehran, Iran
[b] School of Civil Engineering, College of Engineering, University of Tehran, Tehran, Iran

ARTICLE INFO

Keywords:
Machine learning
Genetic algorithm
Structural optimization
Design automation
Reinforced concrete design
Computational design

ABSTRACT

An end-to-end computational platform is presented to automate seismic design, optimize structural configurations, and utilize the resulting data to predict structural parameters—such as section sizes, reinforcement for beams and columns, and overall weight—using machine learning algorithms. The platform adopts a modular framework implemented in Python, ensuring adaptability for optimizing various structural configurations. The process begins with a genetic algorithm module, which iteratively generates code-compliant section sizes and reinforcement for beams and columns based on input design variables, including building configuration. Next, a machine learning module utilizes the design outputs to predict structural model parameters. A comprehensive sensitivity analysis was conducted on the parameters of the genetic algorithm to assess their effects on optimization performance. Several machine learning algorithms were evaluated for predicting structural design and weight, with the XGBoost algorithm providing the most accurate predictive results. Additionally, the platform enables the creation of an extensive database of concrete special moment frame buildings, supporting the development of analytics-driven optimized design methods. Several machine learning algorithms including XGBoost, Random Forest, Decision Tree, Support Vector Regression, and K-Nearest Neighbors were evaluated for predicting structural design and weight, with XGBoost emerging as the most robust model. XGBoost outperforms other models by approximately 15–20 % in terms of predictive accuracy on the testing dataset. The smaller performance gap between training and testing datasets further underscores its ability to avoid overfitting. Among all features, the number of stories was identified as the most influential in predicting structural parameters, contributing approximately 60 % to feature importance.

1. Introduction and research background

Despite the widespread adoption of machine learning across various fields, significant barriers persist in structural design, hindering its broader application in this domain. A major issue impeding the use of ML models for optimization and performance simulations is their computational intensity, which makes them less feasible for traditional early design workflows [1]. Another critical challenge is the scarcity of high-quality diverse datasets, which are essential for developing machine learning models capable of

* Corresponding author.
E-mail address: epackachis@aut.ac.ir (S. Epackachi).

https://doi.org/10.1016/j.jobe.2025.112222
Received 11 October 2024; Received in revised form 24 February 2025; Accepted 25 February 2025
Available online 28 February 2025
2352-7102/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

accurate performance and generalization across varied design scenarios [2]. This data scarcity limits model adaptability and prevent their broader adoption/evolvement in industry. However, despite these barriers, the increased adoption of this emerging technology has the potential to bring about a profound transformation in this field. AI-driven solutions may incorporate structural configurations that would not have been possible to discover within the human-defined design space. This highlights one of the key advantages of utilizing AI in design: the potential to explore options beyond those traditionally developed by human intelligence [3–5].

However, the adoption of machine learning-driven solutions is not that much welcomed because of the cultural resistance within the industry. Many engineers are apprehensive about automated approaches that appear to supersede the human intuition and decision-making processes that have traditionally been central to engineering practice. Despite advancements in models like convolutional neural networks and surrogate algorithms, their adoption is hindered by a lack of user-friendly interfaces and a 'black-box' perception. This reduces trust and limits effectiveness, especially for real-time decision-making during early design stages [6,7]. Re-envisioning ML as an enhancement to human creativity rather than a mere automation tool opens opportunities for collaborative human-machine interaction, where designers can leverage heuristics to streamline the search for feasible designs, balancing computational power with human intuition [8].

The optimization of concrete structures has garnered significant research attention, largely driven by economic considerations within the construction industry. For example, Guan et al. [9] developed a Python-based platform that automates the seismic design workflow, from initial configuration to detailed analysis, by creating accurate nonlinear structural models and utilizing advanced computational techniques. The design of three-dimensional reinforced concrete (RC) structures has also been enhanced by integrating optimization techniques. Dehnavipour [10] developed an optimization framework to improve the efficiency and efficacy of 3D RC structural design. Similarly, Ulusoy et al. [11] introduced an optimization framework specifically for multi-story, multi-span RC frame structures subjected to static loads, aiming to minimize material consumption and construction costs while ensuring structural integrity and performance.

Recent advancements in structural design approaches have increasingly emphasized automation and parametric methodologies to enhance efficiency and effectiveness. Rempling et al. [12] developed a set-based parametric design framework that generates a broad range of design solutions based on predefined parameters, systematically narrowing down the options to identify the optimal design. A novel method for optimizing reinforced concrete building frames has also emerged, focusing on the automated grouping of columns based on load-bearing capacities and other pertinent parameters to improve structural performance [13]. In this research, Boscardin et al. developed optimization algorithms that enable more efficient material allocation, ensuring each column meets its specific load requirements without unnecessary overdesign.

Further contributions to structural engineering have been made by integrating advanced computational methodologies. Mangal and Cheng [14] developed a framework that combines Building Information Modeling (BIM) with a hybrid genetic algorithm to optimize steel reinforcement in RC building frames, thereby automating the optimization of reinforcement and enhancing design efficiency and precision. Cazacu and Grama [15] also explored the use of genetic algorithms and particle swarm optimization for optimizing structural systems. Optimizing three-dimensional steel structures is another critical area of research, particularly given the increasing demand for efficient and environmentally conscious designs. Gero's research [16] highlights how genetic algorithms can explore vast design spaces and generate innovative solutions, often outperforming traditional methods in identifying optimal structural configurations. These findings suggest that genetic algorithms provide superior results in terms of design flexibility and performance optimization compared to classical techniques.

In recent years, civil engineering has undergone a significant transformation driven by advancements in data science and computational technologies. One of the most impactful developments has been the integration of machine learning (ML) techniques into structural assessment and civil engineering. Machine learning, a subset of artificial intelligence (AI), involves the use of algorithms and statistical models to enable computers to perform tasks based on data patterns and inference, rather than explicit instructions. These techniques have significantly enhanced capabilities in structural assessment, including real-time monitoring, damage detection, and performance prediction. Vanluchene and Sun [17] pioneered the use of back-propagation neural networks (ANNs) to address building structural design challenges, marking an early application of machine learning in civil engineering. Their work demonstrated that ANNs could successfully model complex relationships in structural design. Mangalathu and Jeon [18] further advanced this field by developing models to predict structural parameters of reinforced concrete components, significantly improving prediction accuracy using machine learning algorithms. Huang and Burton [19] employed classification models to predict failure modes in structural components, a critical aspect of structural health monitoring and maintenance. Mangalathu et al. [20] expanded on these efforts by creating comprehensive models that use both regression and classification techniques, incorporating a variety of machine learning algorithms for robust predictions.

XGBoost (eXtreme Gradient Boosting), proposed by Chen, has become one of the most popular algorithms in machine learning due to its excellent prediction performance and model interpretability [21]. It has been widely applied in construction equipment monitoring [22], structural health monitoring, and construction safety [23]. XGBoost has also been used for preliminary cost estimation of construction projects [24]. In a comparative study, Haytham [25] applied 20 different AI algorithms—including Fuzzy Logic, Artificial Neural Networks, Case-Based Reasoning, Random Forest, and Support Vector Machine—to estimate engineering costs for canal improvement projects. Among these, XGBoost proved to be the most accurate and applicable.

Support Vector Machines (SVMs) are also employed for regression and classification tasks, demonstrating robustness in handling various data types [14]. Random Forests (RFs) are particularly effective for classification tasks in structural health monitoring and failure mode prediction [22,23]. Overall, machine learning techniques have greatly improved the prediction accuracy of structural responses and performance, proving invaluable for interpreting experimental data, optimizing design parameters, and enhancing the efficiency of structural assessments [26].

While numerous studies have focused on enhancing the efficiency of concrete structures, there is a notable gap in the literature regarding the integration of design guidelines and construction challenges into the optimization process and as well as the provision of reliable data for machine learning models. Optimization algorithms can help address the challenge of data scarcity in structural design. The data generated through these methods are not only reliable but also more optimized and sustainable. This study aims to develop an approach for optimizing concrete structures, ensuring compliance with design restrictions while meeting performance and geometric criteria, and simultaneously establishes a platform for generating reliable data. To achieve this, designs and analyses were conducted following standard structural design processes and in compliance with the latest building codes. ETABS software was utilized for designing reinforced concrete (RC) moment frame structures due to its widespread use in structural analysis and design. The objective of this study is to develop an algorithm that leverages machine learning techniques for the design of mid-rise, short-to-medium span structures. When the structural system is developed concurrently and in coordination with the architectural elements, it can guide the architects' decisions and promote a more unified design process [12]. Conversely, neglecting structural design during these initial phases can result in delays, conflicts between architects and engineers, and undesirable compromises [27]. Furthermore, the results contribute to sustainable construction practices by promoting material efficiency and resource conservation.

This study examines 54 residential buildings with special RC moment frame systems, considering buildings with 4, 6, and 8 floors, and various span configurations including 3, 5, and 7 spans of 4, 6, or 8 m in length. Due to study constraints, all structures were assumed to be symmetrical with equal span lengths throughout. The structural analysis assumptions and seismic details are based on the requirements of ASCE 07–22 [28] and ACI 318-19 [29]. The workflow diagram presented in Fig. 1 illustrates the sequential steps involved in optimizing and implementing machine learning algorithms, providing a clear visual representation of the process flow. This diagram serves as a valuable tool for understanding the systematic approach to preparing data for machine learning.

1.1. Formulation of optimization problem

Optimization algorithms in engineering design aim to find the best compromise among several often conflicting requirements. In the design of concrete structures, optimizing the dimensions of members and the number and arrangement of reinforcing bars are crucial for economic efficiency. This presents a challenge for engineers in selecting the most suitable sections and reinforcement layouts based on maximum member capacity.

Extensive research has been conducted in the area of structural optimization, primarily focusing on two-dimensional models and

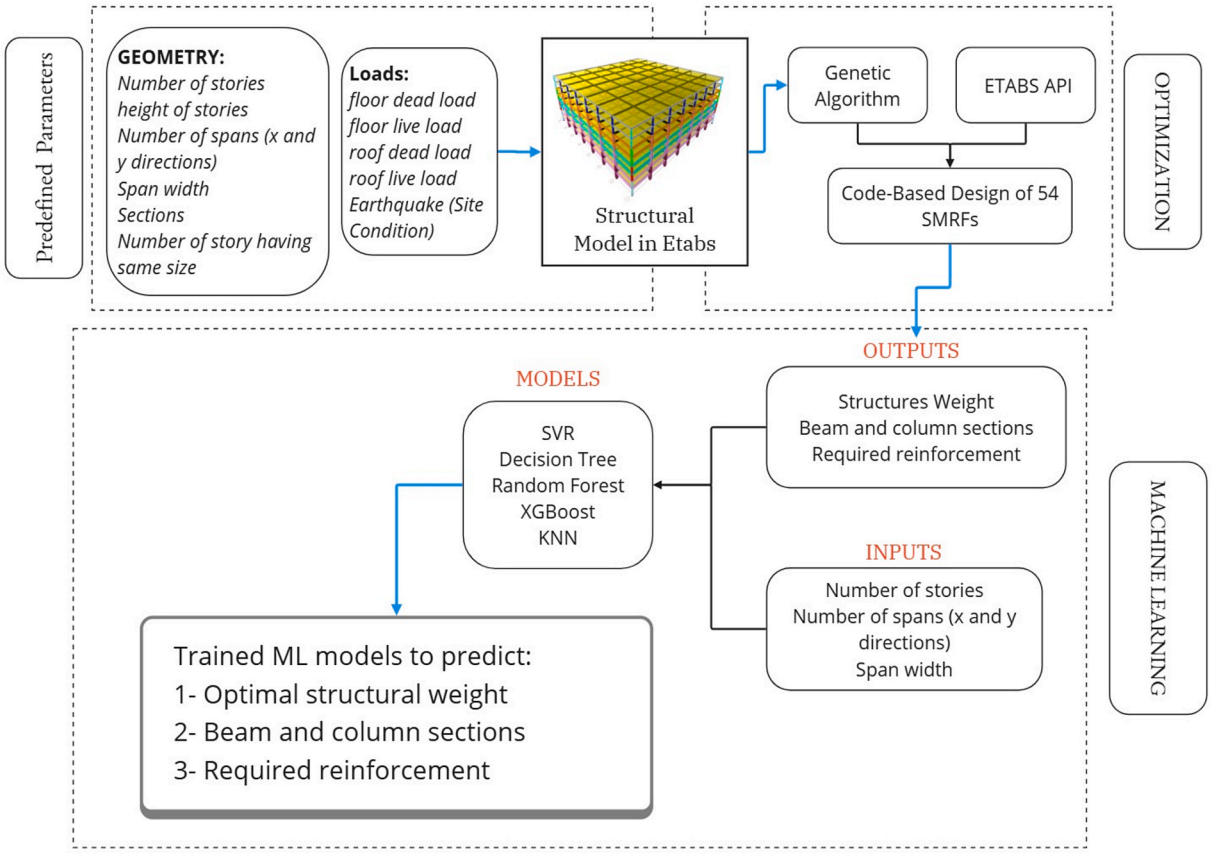


Fig. 1. Workflow diagram illustrating the sequential steps involved in optimizing and implementing machine learning algorithms.

utilizing databases containing member characteristics [30]. However, this approach has two main limitations: it may rely on assumed features due to time constraints, and limited databases may not account for all optimal solutions, leading to local rather than global optimization.

A typical optimization problem generally consists of three main components: (1) the design variables, (2) the objective function, and (3) the constraints. The objective function, which is the quantity to be minimized or maximized, can be represented in general form as shown in Eq. (1).

$$\text{Minimize } \mathrm{F(X)}, \boldsymbol{x} = [x_1, x_2, x_3, \ldots, x_n]^T \tag{1}$$

$$\text{subject to}: \begin{cases} G_m(x) \leq 0 & , \quad m = 1, 2, 3 \ldots o \\ h_n(x) = 0 & , \quad n = 1, 2, 3 \ldots p \\ X_l^{min} < X_l < X_l^{max} & , \quad l = 1, 2, 3 \ldots q \end{cases}$$

where x represents a vector of n design variables, the objective function F(X) computes a scalar value that needs to be minimized. This scalar value could represent various factors such as cost, total frame stiffness, or weight. The vector function $G_m(x)$ & $h_n(x)$, define the non-equivalent and equivalent constraints of the problem, which are considered based on factors such as seismic and constructional requirements, respectively. X_l^{min} and X_l^{max} are the lower and upper bounds of variables, respectively.

1.2. Objective function for 3D reinforced concrete structures

In constrained optimization problems, the initial focus of the objective function is typically on minimizing certain quantities, such as cost or weight. For three-dimensional reinforced concrete (RC) structures, the objective function extends beyond simply minimizing the weight of the materials. Both concrete and rebar are essential components that contribute to the overall weight and cost of the structure. Thus, it is necessary to consider these components separately within the objective function due to their distinct contributions and cost implications.

In structural design, material costs play a crucial role in determining overall expenses. However, focusing solely on weight minimization does not accurately represent costs, especially when the unit prices of materials such as concrete and rebar vary. Therefore, it is essential to incorporate a price effect coefficient into the objective function to ensure it accurately reflects the true structural costs. While the objective function may primarily target minimizing the structure's weight, it ultimately aims to reduce the overall cost by including specific coefficients and parameters. To calculate these coefficients, estimates of the total cost per unit weight for concrete and rebar are obtained. The overall cost is then determined by applying the cost rate to the final weight of the reinforcing bars, as represented in the general form in Eq. (2).

$$Weight = W_c + (cost\ rate \times W_s) \tag{2}$$

In the above equation, the parameters of W_c, W_s are concrete and steel weights, respectively. The term "cost rate" accounts for the relative cost of steel compared to concrete, ensuring that the weight of the structure, which directly impacts material costs, is accurately reflected in the optimization process. This weight equation ties into the objective function F(X) presented in Eq. (1), which is formulated to minimize the total cost of the structure, including both material and construction costs.

1.3. Optimization variables

In the optimization framework, the vector X presented in Eq. (1) consists of design variables, specifically the cross-sectional dimensions of the members, such as beam and column sizes. While the optimization process primarily focuses on these dimensions, the effects of reinforcement quantities are incorporated into the objective function to ensure a comprehensive evaluation of structural performance. To address the continuous nature of member dimensions, these variables are discretized by incorporating sections with restricted step increments. The reinforcement area is treated as a continuous variable. Two approaches are considered for handling reinforcement: (1) Predetermined Member Dimensions, where the reinforcement amount is maintained based on predefined member dimensions, and (2) Design Software Integration, in which alternatively, design software provides the required reinforcement quantity to the algorithm for weight computation. For columns, both the reinforcement bars and their positions are predetermined. For beams, the required bending and shear reinforcement are calculated using design software and subsequently supplied to the algorithm. The list

Table 1
Beam and column sections.

Column Sections			*Beam Sections*	
30 × 30-4 Φ 20	50 × 50-12Φ20	75 × 75-20 Φ 32	30 × 30	40 × 60
30 × 30-8 Φ 16	50 × 50-20 Φ 20	80 × 80-24 Φ 25	30 × 40	50 × 50
40 × 40-4 Φ 20	50 × 50-24 Φ 20	85 × 85-26 Φ 32	30 × 50	50 × 60
40 × 40-8 Φ 20	60 × 60-20 Φ 25	90 × 90-28 Φ 32	40 × 40	50 × 70
50 × 50-12 Φ 16	75 × 75-20 Φ 25		40 × 50	50 × 80

of beam and column sections is detailed in Table 1.

1.3.1. Grouping of beams and columns

To simplify the optimization process, members with similar force characteristics are grouped. Beams are categorized into outer and central groups, while columns are classified as corner, side, and middle. This grouping strategy aligns with conventional design practices and helps reduce optimization time. It has also been used in previous studies [13]. The subdivision of members is based on load distribution and the influence of key structural elements on stiffness, which is crucial for drift control. Additionally, the grouping of beams and columns is varied every two stories to account for changes in structural behavior and loading conditions, as illustrated in Fig. 2.

Therefore, 20, 15, and 10 groups were generated for 8, 6, and 4-story structures, respectively. The total number of groups for each type of building is presented in Table 2.

In this study, a novel approach for variable assignment is utilized. Rather than assigning a single set of dimensions to all columns, dimensions are assigned based on the vertical position of the columns, recognizing that columns on upper floors often have different dimensions than those on lower floors. To accommodate this variation, columns are classified into three dimension-based groups: small, medium, and large. These groups are then strategically assigned to lower, middle, or upper floors, corresponding to their roles as corner, side, or middle columns.

This approach reduces the number of variables, which significantly decreases the optimization time and enhances computational efficiency. An example of this grouping strategy is shown in Table 3.

1.3.2. Constraint handling

In the optimization process, it is essential to account for specific constraints to ensure safety and reliability. Constraints on the variables are imposed by the $G_m(x)$ term, presented in Eq. (1), ensuring the final design complies with structural and code-based requirements. In this research, there are no equality constraints $h_n(x)$; instead, a penalty function is utilized to effectively handle the constraints within the optimization framework. The constraints considered in this study are divided into two main categories: (1) design constraints and (2) geometric constraints. Design constraints are the mandatory requirements for special reinforced concrete moment frame systems as specified by ASCE 07–22 [28] and ACI 318-19 [29], as summarized in Table 4. Design constraints are critical for ensuring compliance with structural standards and safety regulations. Geometric constraints, on the other hand, pertain to the physical dimensions and proportions of structural elements, ensuring the feasibility of the design from a geometric standpoint. By incorporating both design and geometric constraints, the optimization process adheres to established safety and performance standards, resulting in a robust and reliable structural design.

Fig. 3 presents a flowchart that categorizes the constraints involved in the design process into two main types: design constraints and geometric constraints. Each of these categories is further subdivided into specific constraints for beams, columns, and joints, providing a detailed overview of the considerations necessary for effective structural design. In this figure, "PMM" represents the P-M-

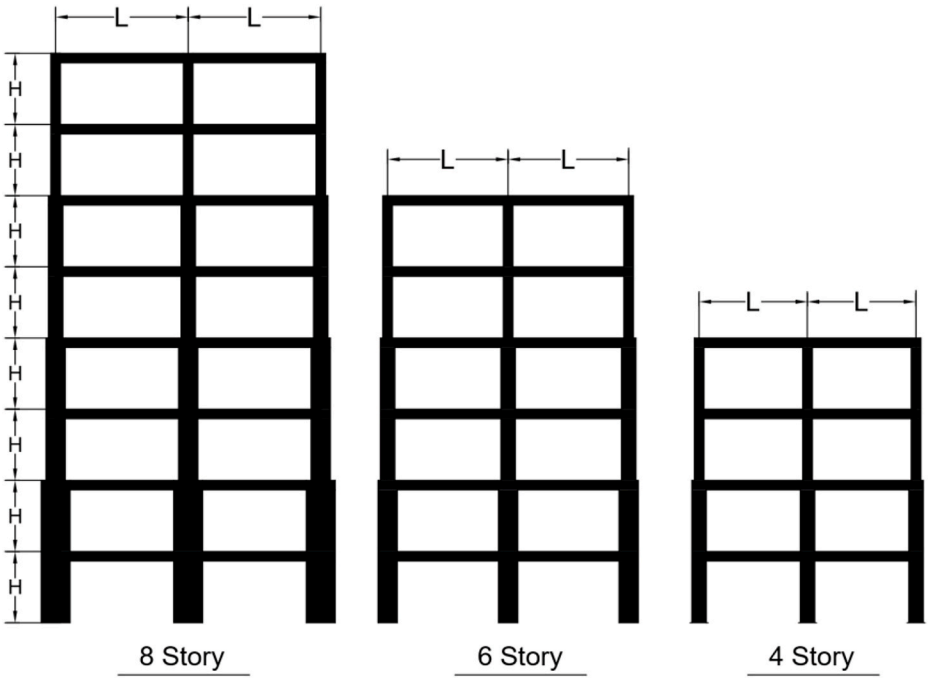


Fig. 2. Column grouping strategy (every 2 stories).

Table 2
Number of beams and columns group.

	8 story	6 story	4 story
Beams group	8	6	4
Column group	12	9	6
Total No. of groups	**20**	**15**	**10**

Table 3
A sample column grouping.

column group 1	column group 2	column group 3
30 × 30-4ϕ20	40 × 40-8ϕ20	50 × 50-12ϕ20
40 × 40-4ϕ20	50 × 50-12ϕ16	50 × 50-20ϕ20
40 × 40-8ϕ20	50 × 50-12ϕ20	50 × 50-24ϕ20
50 × 50-12ϕ16	50 × 50-20ϕ20	60 × 60-20ϕ25
50 × 50-12ϕ20	50 × 50-24ϕ20	75 × 75-20ϕ25

Table 4
Design restrictions according to ACI 318-19.

Element	Control Item	Design constraints		Geometry constraints
Column	(P-M2-M3)	ACI[a] 10.4.2	ACI 18.7.3	ACI 10.3.1
		ACI 10.5.1	ACI 18.7.4	ACI 18.7.2
		ACI 10.5.2	ACI 22.2	
		ACI 10.6.1		
	Shear & Torsion	ACI 10.5.1	ACI 18.7.5	
		ACI 10.5.3	ACI 18.7.6	
		ACI 10.5.4	ACI 22.5	
		ACI 10.6.2	ACI 22.7	
		ACI 10.7.6		
Beam	Flexure	ACI 9.4.1	ACI 9.6.2	ACI 9.3.1
		ACI 9.4.2	ACI 9.7.3	ACI 9.3.2
		ACI 9.5.2	ACI 18.6.3	ACI 18.6.2
		ACI 9.6.1	ACI 22.3	
	Shear & Torsion	ACI 9.4.3	ACI 9.7.5	
		ACI 9.4.4	ACI 9.7.6	
		ACI 9.5.3	ACI 18.6.4	
		ACI 9.5.4	ACI 18.6.5	
		ACI 9.6.3	ACI 22.5	
		ACI 9.6.4	ACI 22.7	
Joint	Beam-Column Capacity (BCC)	ACI 18.7.3		ACI 18.8.2.3
	Joint shear	ACI 18.8.4		

[a] All ACI numbers refer to the Sections of the ACI 318-19 standards [29].

M interaction ratio for columns, while "BCC" denotes the beam-column capacity ratio.

Consequently, the cumulative values of these parameters determine the overall performance of the entire structure. For instance, the overall violation of PMM and drift ratio can be calculated as Eq. (3) and Eq. (4).

$$PMM\ RATIO = \sum_{i=1}^{No.\ Col} PMM_i \tag{3}$$

$$DRIFT\ RATIO = \sum_{j=1}^{No.\ Story} Drift_j \tag{4}$$

2. Methodology: Description of the genetic algorithm

The genetic algorithm (GA) is inspired by natural selection and the evolutionary processes observed in nature. Introduced by J. Holland in the 1970s [31], genetic algorithms leverage the concepts of gene generation, integration, and the dominance of advantageous traits to find optimal solutions. In a GA, the solution space is represented by a population of individuals, each encoding a potential solution to the optimization problem. The algorithm evolves this population iteratively, aiming to discover the fittest individuals through successive generations. The evolution process begins with an initial population, randomly generated within the solution space, and involves applying genetic operators—such as selection, crossover, and mutation—to create a new population.

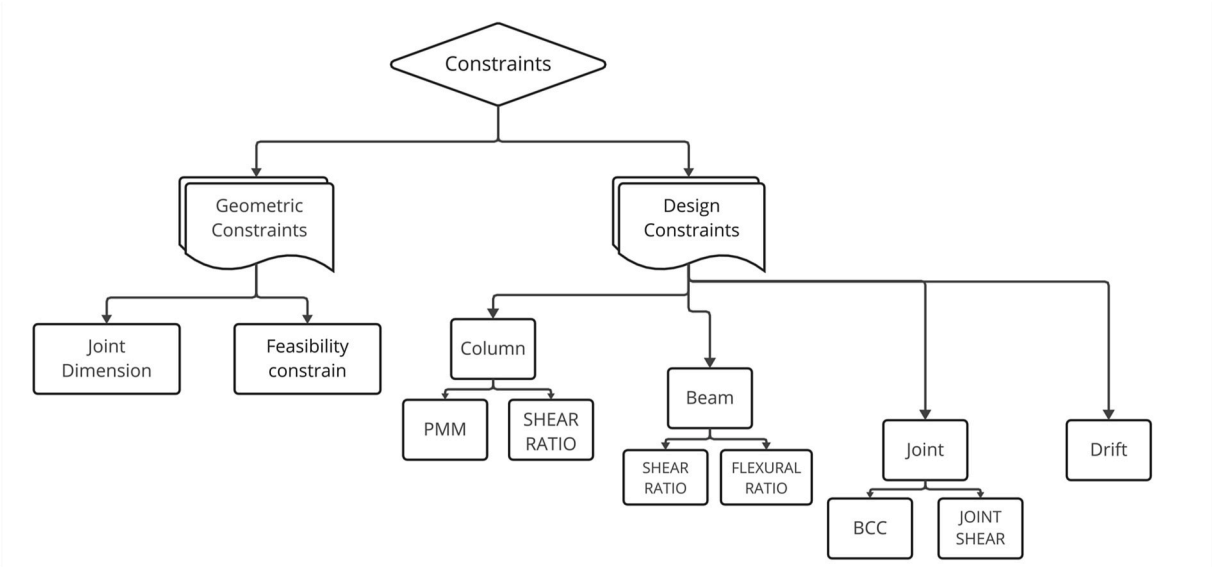


Fig. 3. Design constraints.

Fig. 4 illustrates the simplified process of the genetic algorithm, highlighting its key components and flow. Each individual in the population is encoded as a chromosome, which is a collection of genes. Each gene represents a parameter of the problem to be optimized. The fitness of each individual, which indicates the quality of the solution, is evaluated using a fitness function that quantitatively measures how well the solution meets the problem's criteria [32].

The GA iteratively refines the population by selecting individuals based on their fitness, recombining them to produce offspring, and introducing mutations to maintain diversity. This process continues for a predetermined number of generations or until a satisfactory solution is achieved [15]. Through this approach, the genetic algorithm efficiently explores the solution space, converging towards an optimal or near-optimal solution by mimicking natural evolutionary processes.

2.1. Genetic operators

Genetic algorithms (GAs) employ several genetic operators, such as selection, crossover, and mutation, to create a new population in each iteration. These operators simulate natural evolutionary processes, guiding the population toward optimal solutions.

The selection operator is fundamental in a genetic algorithm, as it preferentially selects individuals with higher fitness scores to serve as parents for the next generation. This mimics natural selection, where the fittest individuals are more likely to reproduce and pass on their advantageous traits. Various selection methods can be applied, including Roulette Wheel Selection, Stochastic Universal Sampling, and Tournament Selection. Among these, Tournament Selection generally outperforms Roulette Wheel Selection in terms of convergence speed and robustness, as it emphasizes stronger candidates, leading to potentially higher fidelity. In contrast, Roulette Wheel Selection allows for a broader exploration of the solution space but may be less consistent in maintaining high fidelity compared to Tournament Selection. The quicker convergence of Tournament Selection-based genetic algorithms typically results in more efficient problem-solving [33,34].

Elitism is a technique used in genetic algorithms to ensure that the best individuals from the current generation are preserved and carried over to the next generation. Typically, only a portion of individuals, known as the Gap Generation—ranging between 80 % and 90 % [35,36]—is selected to advance. When only 80 % of individuals are transferred, the remaining slots in the next generation may be filled by randomly selecting individuals, which can enhance the algorithm's ability to explore new solutions (a concept referred to as "juvenility"). This approach helps maintain diversity in the population and prevents premature convergence to suboptimal solutions.

The crossover operator is another crucial mechanism in genetic algorithms. It combines the genetic information of two parent solutions to produce new offspring, promoting genetic diversity within the population. Various crossover techniques can be employed, such as one-point, two-point, and uniform crossover. Uniform and multi-point crossover methods are considered effective due to their high search capabilities, which enable the exploration of diverse combinations. In contrast, one-point and two-point crossover methods generally offer less exploration compared to uniform crossover but can still be useful for combining beneficial traits from parent solutions.

The mutation operator plays a vital role in genetic algorithms by introducing random changes to solution strings. This operator increases the convergence speed of the algorithm and helps escape local optima, enhancing the likelihood of reaching a global optimum solution. While mutations can potentially worsen some solutions, the natural selection process eliminates these poorer solutions in subsequent generations, ensuring that the overall quality of the population improves over time.

By effectively utilizing these genetic operators—selection, elitism, crossover, and mutation—genetic algorithms can efficiently

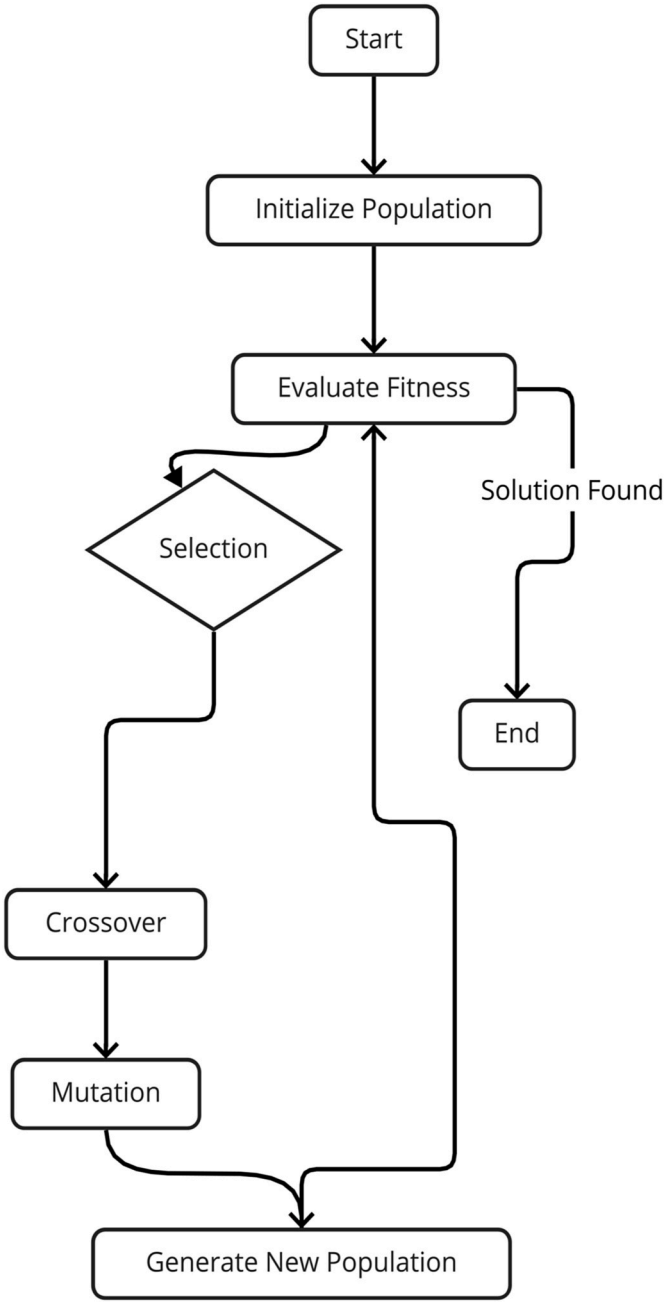


Fig. 4. Genetic algorithm flowchart.

navigate the solution space, evolving toward optimal solutions while maintaining diversity and avoiding premature convergence.

2.2. Penalty function and coefficient

Penalty functions are employed in optimization processes to effectively manage constraints. These functions are integrated into the objective function to penalize violations of the constraints, thereby transforming a constrained problem into an unconstrained one. The penalty function is calculated by multiplying a penalty coefficient by the sum of constraint violations, making it a vital component of the objective function in constrained optimization problems. The penalty coefficient is a key factor in optimization algorithms, such as genetic algorithms, that helps balance exploration and exploitation. Initially, a lower penalty coefficient is applied to encourage extensive exploration during the search phase. As the algorithm progresses, the penalty coefficient is gradually increased, allowing the algorithm to focus more on exploitation and finding an optimal solution that meets the problem's constraints and requirements.

In this research, a new method was implemented, involving a gradual linear increase of the penalty coefficient across generations, as formulated in Eq. (5). This approach enhances the algorithm's ability to adapt and optimize effectively over time.

$$PF = a \times \frac{i^{th}\ generation}{max\ generation} + b \tag{5}$$

In this equation, "a" is a constant factor that regulates the rate of increase of the penalty coefficient, allowing for a controlled adjustment over the generations. The "b" constant serves as a baseline value for the penalty coefficient, ensuring that it starts from an appropriate initial level. This approach enhances the algorithm's ability to adapt and optimize effectively over time.

2.3. Constraint normalization

Normalization of constraints, variables, and the fitness (objective) function is crucial in constrained optimization. It facilitates fair comparison across varying units and values, eliminates unit dependencies, and ensures accurate assessment of violations against normalized constraints.

The general approach to constraint normalization can be expressed as shown in Eq. (6).

$$\frac{\sum \left(\frac{constrain - allowable}{allowable} \right)^2}{No.\ element} \tag{6}$$

For instance, when considering column interaction constraints and floor drifts, the normalization is applied as Eq. (7) and Eq. (8).

$$\frac{\sum \left(\frac{PMM\ Column - 1}{1} \right)^2}{No.\ Columns} \tag{7}$$

$$\frac{\sum \left(\frac{Drift - Allowable\ Drift}{Allowable\ Drift} \right)^2}{No.\ Story} \tag{8}$$

Finally, the pseudo-normalized objective function is presented as Eq. (9).

$$Pseudo\ Fitness\ Function = \frac{weight}{mean\ weight} + PF \times \frac{\sum Constrains}{No.Constrains} \tag{9}$$

To normalize the weight, an estimate of the average weight of structures with specific aspect ratios and heights is required. The mean weight is determined by calculating the average of the weights of a structure with the smallest dimensions and a structure with the largest dimensions within the section group. This methodology ensures that the normalization process appropriately reflects the range of dimensions and provides a consistent basis for comparing different structures.

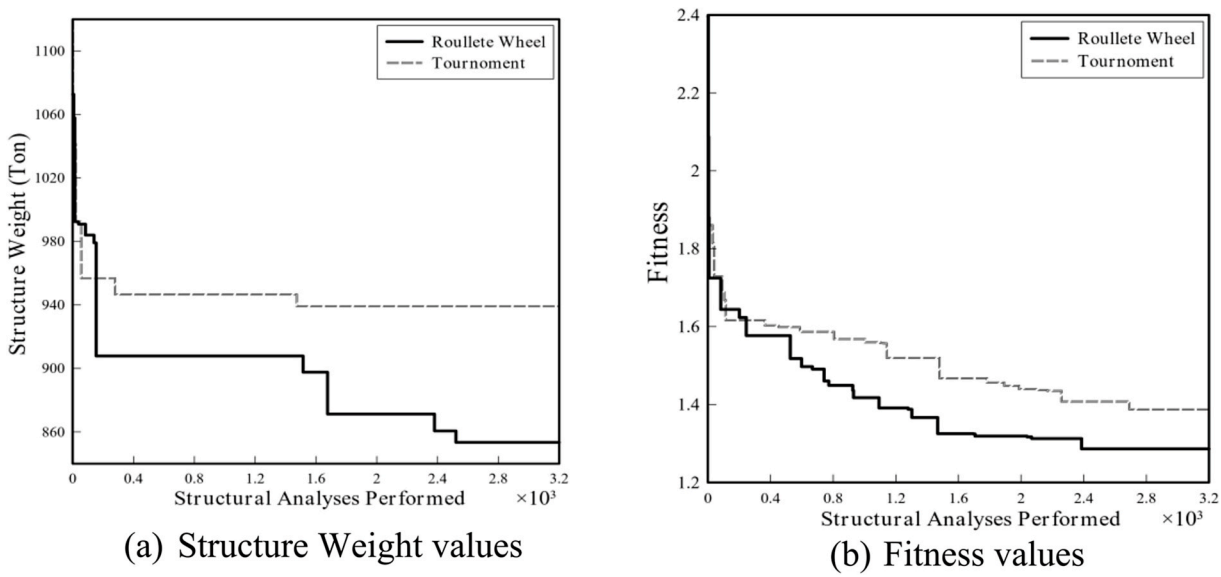


Fig. 5. Convergence histories for Roulette Wheel and Tournament selection methods.

3. Optimization algorithm implementation

The optimization algorithm implementation involves defining genes and encoding chromosomes within a genetic algorithm to represent structural sections for columns and beams. Each chromosome encodes the solution as a sequence of numbers, with genes corresponding to different structural groups. The algorithm generates and evaluates beam and column sections using ETABS structural analysis software, connected through its API for automated data exchange. The optimization process is implemented using the DEAP library [37,38] in Python, chosen for its extensive features, support for parallelism, and ease of use. This setup allows for efficient optimization and integration with other Python libraries for data processing and machine learning [39].

3.1. Parameter optimization

Determining the optimal parameter values for the genetic algorithm requires sensitivity analysis. Since genetic algorithms are stochastic methods, results may vary between different runs. This section focuses on identifying the most effective parameter settings by systematically adjusting the algorithm's parameters. The parameter ranges selected for this study are consistent with those reported in similar research [16].

3.1.1. Selection method

In this study, Roulette Wheel Selection and Tournament Selection are utilized as the primary methods for selection. Fig. 5 presents the results of these selection methods under consistent conditions, comparing their performance and impact on the optimization process.

The analysis reveals that the Roulette Wheel selection method surpasses the Tournament selection method in performance. The structure optimized using the Roulette Wheel weighs 853 tons, whereas the Tournament method results in a weight of 939 tons. Furthermore, the Roulette Wheel method achieves faster convergence (for Roulette Wheel 7.5 h and Tournament 9.5 h, using a system equipped with an Intel Core i7 (8th Gen) processor and 8 GB of RAM) and a closer approximation to the optimal solution, while the Tournament method is prone to becoming trapped in suboptimal local solutions. Therefore, the Roulette Wheel selection method was chosen for this study. This conclusion aligns with prior research, indicating that tournament selection is better suited for smaller problems, while rank-based roulette wheel selection is more effective for larger ones. The choice between these methods involves a trade-off: rank-based selection provides higher solution quality but requires more computation time, making it ideal when quality is prioritized over speed [40].

3.1.2. Population

The optimal size of the genetic algorithm population is influenced by the complexity of the problem and the number of generations. Evaluating the effects of population size on convergence speed and solution quality is essential for achieving an effective balance. A thorough analysis helps in determining the population size that optimally balances computational efficiency with solution accuracy.

Fig. 6 shows the optimal design results for structures with population sizes of 150, 200, 300, and 400. The weights of the optimal structures are 429, 410, 413, and 409.8 tons, respectively, demonstrating minimal variation. The analysis reveals that the structure with a population size of 200 reached its optimal weight after approximately 5500 structural analyses, while the structure with a

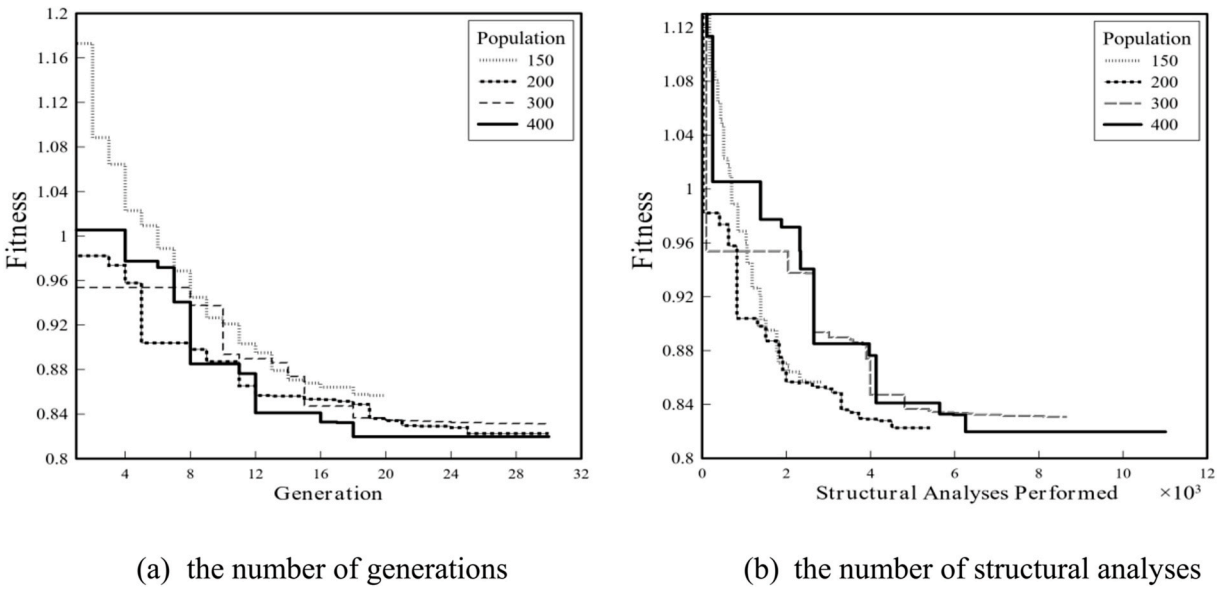


Fig. 6. Convergence histories for roulette wheel and tournament selection methods: (a) structure weight values, (b) fitness values.

population size of 400 achieved a similar weight but required roughly twice as many analyses. Consequently, a population size of 200 was selected for this study.

3.1.3. Mutation

The mutation process in a genetic algorithm is controlled by the mutation rate, which specifies the probability of a gene being mutated. Higher mutation rates increase the randomness in the algorithm's behavior. Sensitivity analysis evaluates the impact of varying mutation probabilities on the algorithm's performance, allowing for an isolated assessment of each gene's influence. Previous studies suggest that mutation rates are typically set between 3 % and 15 %. In this study, adjustments were made within this range, and the effects on the algorithm's performance were analyzed, as shown in Fig. 7.

The results indicate that a mutation rate of 5 % achieved the most optimal fitness of 0.78. Lower mutation rates (3 %) led to insufficient exploration of the solution space, while higher rates (15 %) resulted in slower convergence and increased constraint violations. Thus, a mutation rate of 5 % was identified as the most suitable for this application. The key parameters of the genetic algorithm used in this study are summarized in Table 5.

3.2. Case studies

The optimization platform is designed to optimize concrete frame structures with varying geometrical configurations. Due to the computational burdens, this study focuses on optimizing structures with 4, 6, and 8 floors. To account for a diverse range of conditions, structures with different aspect ratios were analyzed. The structures considered feature spans of 4 m, 6 m, and 8 m. These spans are configured in several patterns in plan: square configurations of 7 × 7 m, 5 × 5 m, and 3 × 3 m; and rectangular configurations of 7 × 5 m, 5 × 3 m, and 7 × 3 m.

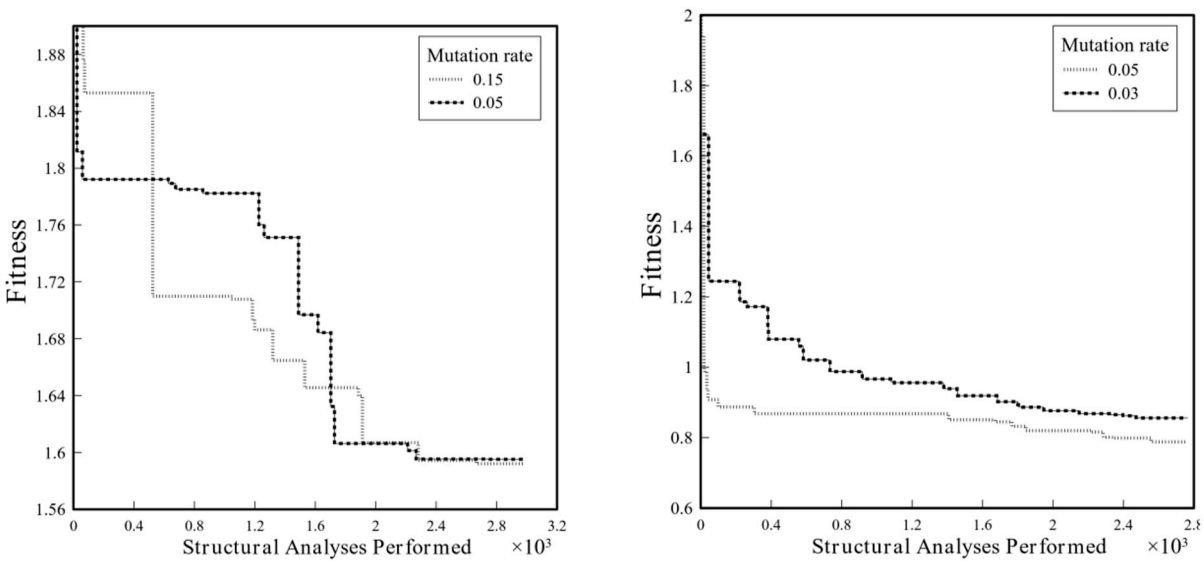
For instance, an eight-story structure was evaluated across 18 different configurations, varying in geometry, number of spans, and span lengths. When including the 4, 6, and 8-floor structures, the total number of analyzed configurations amounts to 54.

As a specific example, Fig. 8 presents a symmetrical 8-story structure with a floor plan of 7 spans, each 8 m long in both directions. The structure is subjected to lateral loads based on the equivalent static method and includes standard gravity loads for residential buildings, such as uniform service dead and live loads on floors. Stiffness modifiers for beams and columns are applied according to the guidelines provided. The frame comprises 1408 components, including 869 beams and 512 columns, organized into 20 groups: 8 beam groups and 12 column groups. Fig. 9 illustrates the layout of these beam and column groupings.

The gravity and earthquake load combinations are based on the provisions of ASCE 7–22 [28].

3.3. Seismic response coefficient

In this study, the seismic coefficient is recalculated for each analysis due to potential changes in the analytical period resulting from modifications to the sections. Although the seismic coefficient can significantly impact structural optimization, it is often overlooked in routine practice. To incorporate period updates into the analysis, a dual-phase approach is required, which extends the optimization



(a) 8-story; 0.05 (20 hours), 0.15(21 hours) (b) 6-story; 0.03 (5.7 hours), 0.05 (7.5 hours)

Fig. 7. Convergence histories for different mutation rate.

Table 5
Summary of GA parameters.

GA parameters	
Selection method	Roulette Wheel
Mutation Rate	5 %
Crossover Method	Uniform
Number of Generation	15
Number of Population	200

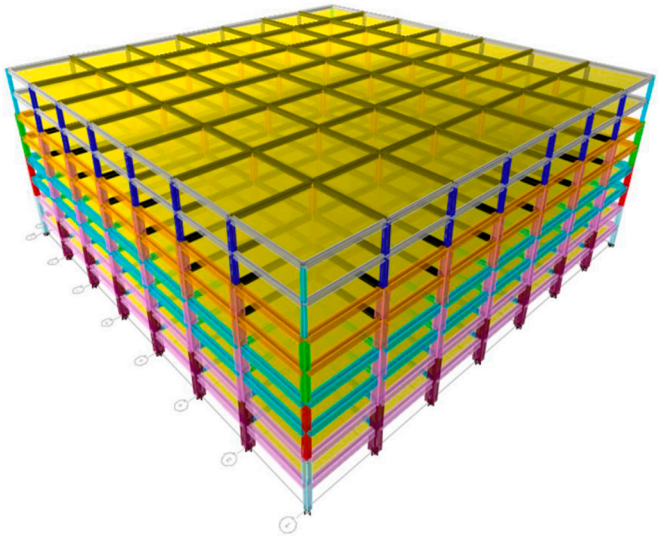


Fig. 8. ETABS model of 8-story, 7 by 7 span structure.

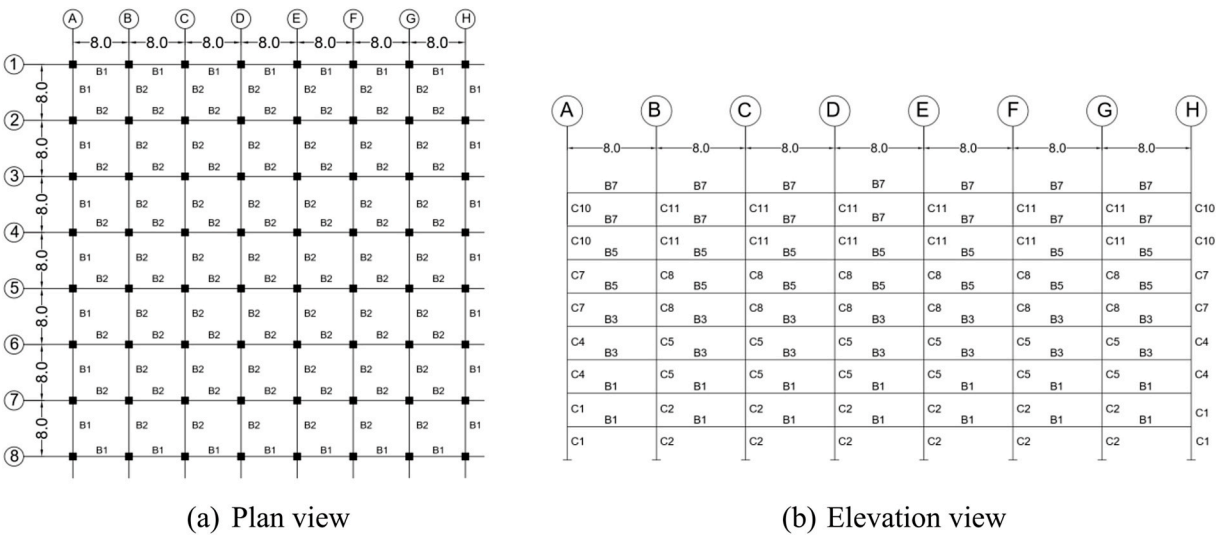


Fig. 9. Member grouping in structure: (a) plan view, (b) elevation view.

time. To mitigate unnecessary time consumption, previously analyzed and designed structures are stored in the algorithm's memory, thereby avoiding redundant analyses and designs when encountering duplicate genes.

The flowchart presented in Fig. 10 summarizing the key steps of the genetic algorithm used for structural optimization. It begins with model preparation, including the calculation of fundamental period and seismic response coefficient, followed by creating an initial population of design solutions. The algorithm iteratively evaluates and modifies the design, checking constraints, calculating fitness, and adjusting groupings of beams and columns to achieve the optimum solution.

3.4. Weight calculation

Construction costs are significantly affected by the weight of reinforcement. While calculating concrete weight is straightforward based on member dimensions, determining the weight of reinforcement for beams and columns involves additional complexities, including rebar hooks, overlaps, and development lengths.

The side view of the beam illustrates the arrangement of longitudinal reinforcement, including both general rebars and additional rebars placed to strengthen critical areas. The longitudinal reinforcement ensures the beam's capacity to resist bending moments, while the additional rebars are strategically placed to enhance strength at regions experiencing higher stress, such as near supports. This detailed layout of reinforcement ensures compliance with structural requirements and improves overall performance under loading conditions, particularly in seismic regions, as shown in Fig. 11. The algorithm details reinforcement at three distinct locations along the beam length. To compute the total number of reinforcing bars, the maximum reinforcement area at the mid-span is considered as the base bar area. Additional bars are added at the start and end points where the reinforcement exceeds this base amount ($As > A_{sgenerai}$). For overlap calculations, an assumption was made regarding the placement of reinforcing bar cuts along the beam length. In this study, a 4-m overlap interval is used for longitudinal reinforcement calculations, presented in Eq. (10)–(12). General Bar Bot

$$L_{gen} = 2 \times (L_{Hook} + L_{dh}) + \sum_{i=1}^{No.\ span} L_i + \left(\frac{\sum_{i=1}^{No.\ span} L_i}{4} \right) \times L_{overlap} \tag{10}$$

$$L_{Add\ bar\ 1} = (L_{Hook} + L_{dh}) + L_{req} + L_d \tag{11}$$

$$L_{Add\ bar\ 2} = L_{req} + 2L_d \tag{12}$$

where the L_{gen} is the total length of general rebars, L_{Hook} is the length of the standard hook, L_{dh} is development length in tension with a standard hook, $L_{overlap}$ is the lap splice length, L_i is the clear length of span in the direction that moments are being determined, $L_{Add\ bar\ 1}$ is the total length of additional rebars with a standard hook, $L_{Add\ bar\ 2}$ is the total length of additional rebars without standard hook, and L_{req} is the length of required additional rebars.

The length of a beam's shear reinforcement, including hoops or stirrups, is calculated based on the beam dimensions and spacing

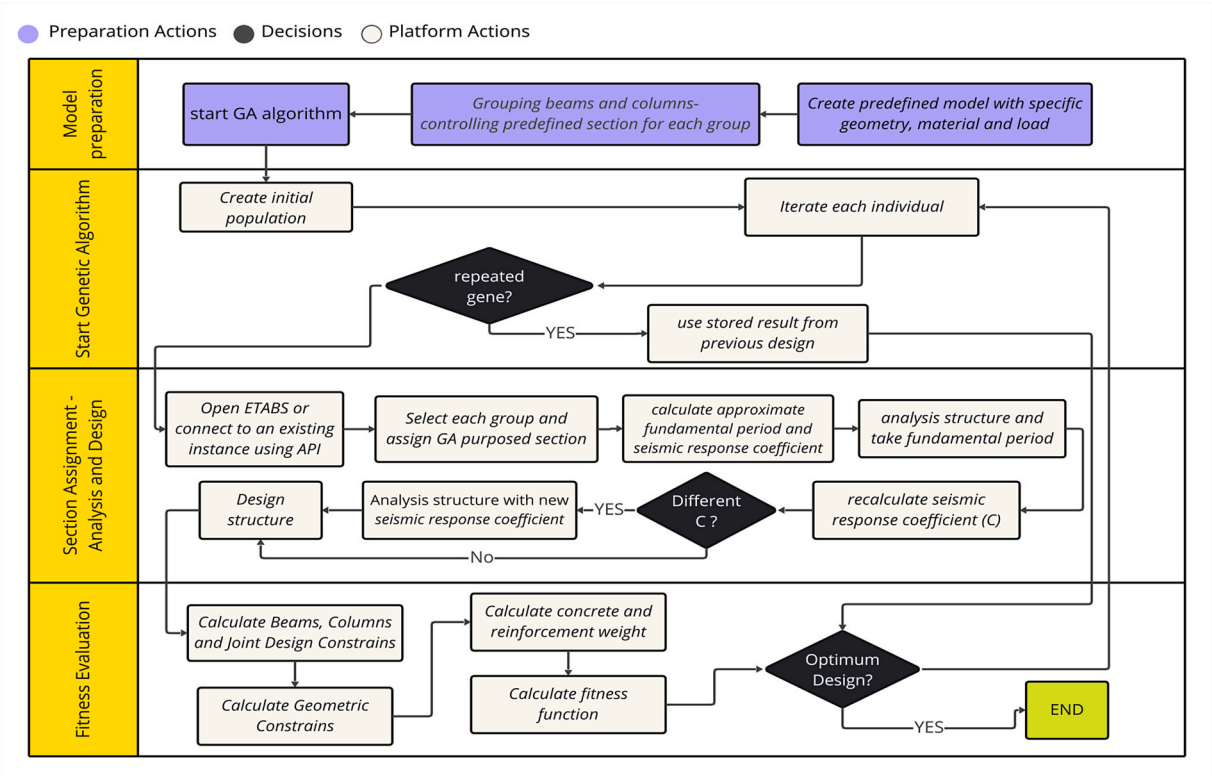


Fig. 10. Proposed framework for automated optimization for RC frames.

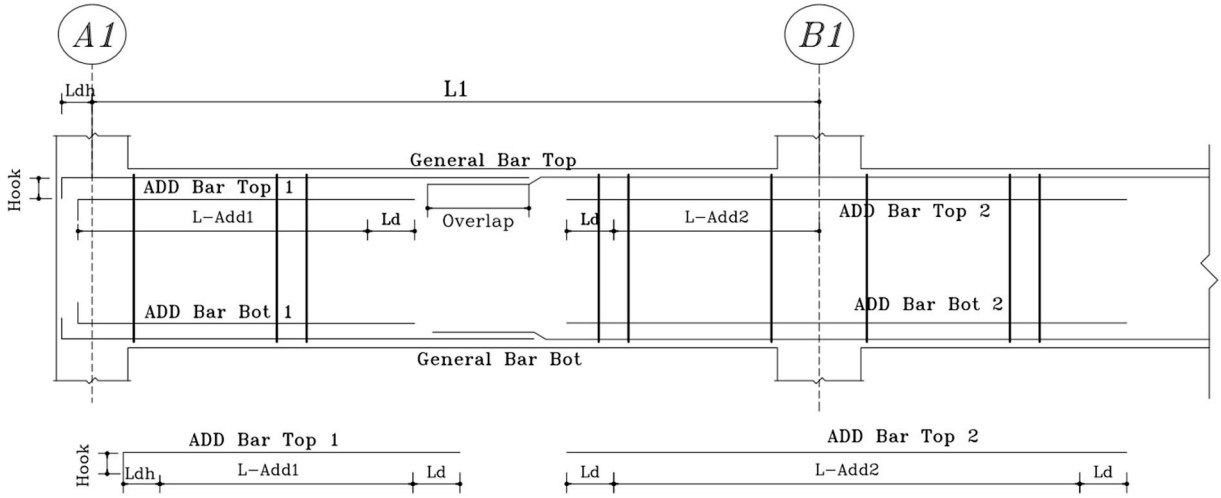


Fig. 11. Side view of a typical beam reinforcement detail.

requirements according to the ACI 318-19 standard [29], some of which are presented in Fig. 12. The total weight of the shear reinforcement is subsequently determined using these calculations.

3.4.1. Column reinforcement weight

The reinforcement for column cross-sections is predefined, and the weight of the rebar is calculated post-optimization once the final cross-sections are established. To quantify the weight of longitudinal reinforcement in columns, the heights of the floors are considered. Typically, a discontinuous reinforcement layout is used, with each floor having a specific reinforcement section.

3.4.2. Stirrups with seismic hooks

This study includes an overlap length for each floor and considers development lengths and hooks for the first and last floors. The weight of the longitudinal reinforcement for the columns is calculated based on these parameters, presented in Eq. (13).

$$W_{CLR} = \sum_{c=1}^{No.\ columns} \left((H_1 + L_3)A_1 + \sum_{i=1}^{No.\ story} (L_{1i} + L_{2i})A_{Li} + H_2 A_2 \right) \times \rho_{steel} \tag{13}$$

Where W_{CLR} is the total weight of column longitudinal reinforcement, H_1 is the length of column hook in foundation, L_3 is the length of longitudinal reinforcement developments into the foundation, A_1 is the area of first longitudinal reinforcements, L_{1i} is the longitudinal reinforcements length (story height), L_{2i} is the length of longitudinal reinforcements overlap, A_{Li} is the area of longitudinal reinforcements, H_1 is the length of the column hook in the roof slab, A_2 is the area of last longitudinal reinforcements and ρ_{steel} is the density of rebars' steel.

The shear reinforcement is calculated following the provisions of ACI 318-19. The distance and number of stirrups are determined based on these provisions, and the total weight of the shear reinforcement is subsequently calculated. Fig. 13 illustrates the transverse

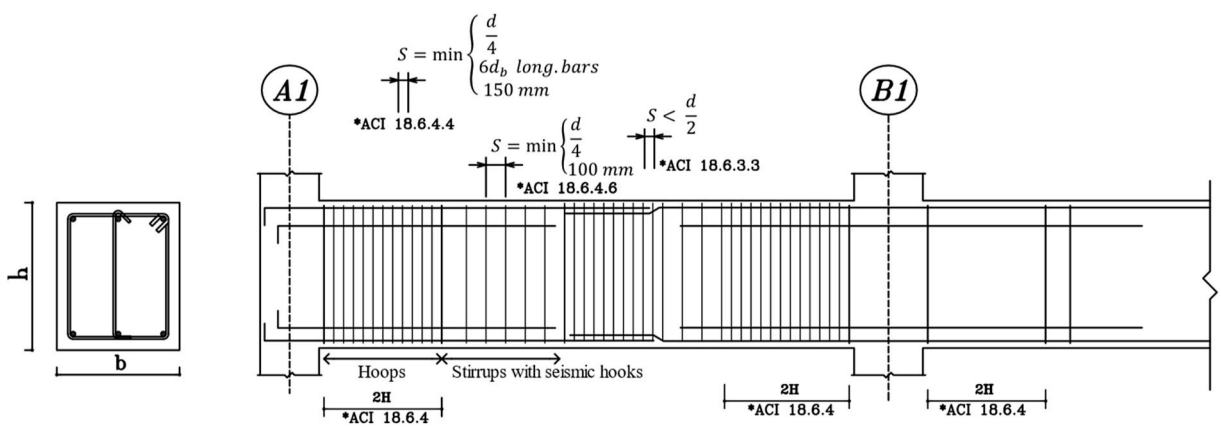


Fig. 12. Shear reinforcement detail of beams.

reinforcement requirements for columns in special moment frames, including the transverse reinforcement required over a length l_0 from each joint face, and on both sides of sections where flexural yielding is likely to occur due to lateral displacements beyond the elastic range. The figure also depicts the spacing requirements for transverse reinforcement in different parts of the column, as specified for ensuring ductile performance under seismic loads.

3.5. Results of optimization process

The optimization of a symmetrical 8-story structure, with a floor plan consisting of 7 spans of 8 m in each direction, required approximately 48 h and involved 3000 structural analyses. The optimal design achieved a minimum weight of 15,865 tons. Fig. 14 illustrates the convergence histories of the best design for this symmetrical 8-story structure with 7 bays of 8 m.

To verify the accuracy of the optimized structures, stress ratios were calculated for a representative optimized structure. The stress distribution and magnitude within this structure are illustrated in Fig. 15. Additionally, Fig. 16 shows the drift distribution along the height of the structure, with the maximum drift approaching the drift limit (elastic drift limit divided by the displacement magnification factor).

3.6. Design with machine learning methods

This section presents the results of applying machine learning models to optimize the design of RC moment frames. The objective was to evaluate the models' accuracy, computational efficiency, and predictive capability in achieving optimal structural configurations. The machine learning models utilize the structures' weight, beam and column sections, and required reinforcement for each section as outputs, along with the number of stories, number of spans in both directions, and span widths as inputs, to develop predictive models. Fig. 17 illustrates the results for the optimal weights across the database, highlighting various configurations and story counts, including 4, 6, and 8 stories. These optimized structural designs, derived from the genetic algorithm, provide key insights into the most efficient configurations. This data serves as the foundation for the subsequent machine learning process, which is employed to develop predictive models aimed at further optimizing structural designs.

The dataset comprised 54 design scenarios, the input variables for the machine learning models include critical structural parameters such as the total number of floors, which influence the overall height and load; the number of spans in both longitudinal and transverse directions, affecting the structural layout; and span lengths, which impact the design dimensions of beams and columns. By leveraging the optimization results, the machine learning models are trained to predict the most efficient design configurations based on these key parameters, enhancing the efficiency and integration of the overall design process.

The machine learning models generate several key outputs, including the optimized weight of the structure, which reflects material efficiency; the dimensions of columns and beams for each group, optimized for safety and performance; and the quantity of reinforcement required for each group, essential for ensuring structural integrity and resilience, especially in seismic areas. These outputs enable engineers to make informed decisions to enhance design processes and achieve optimal structural performance.

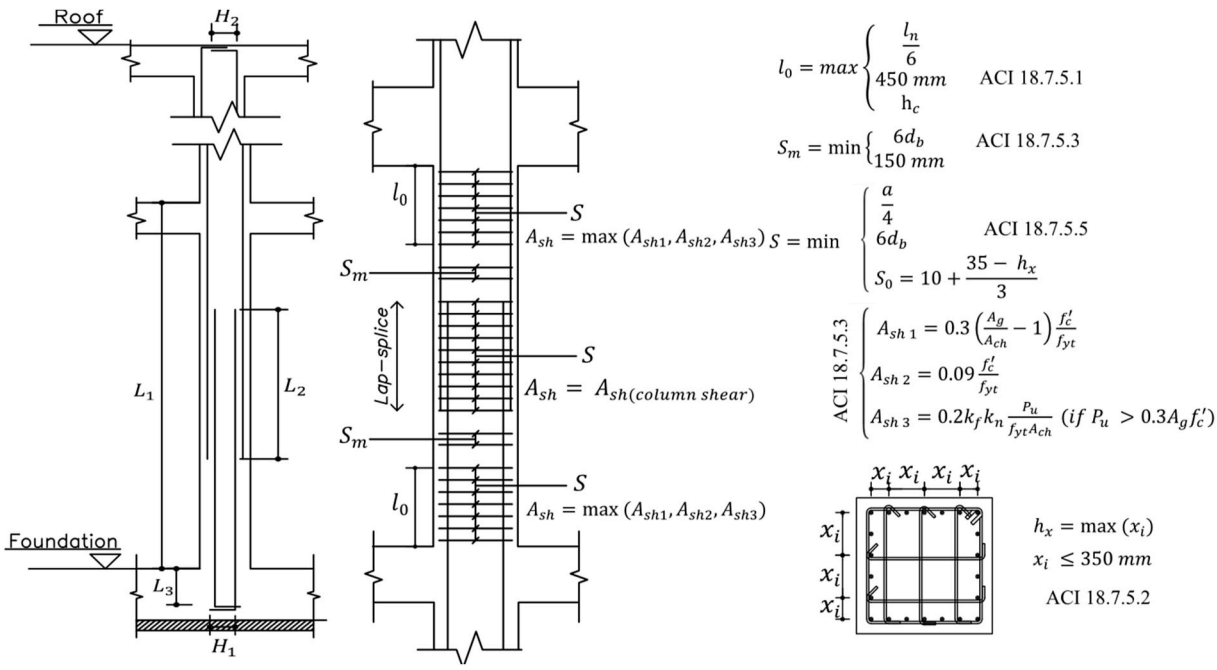


Fig. 13. Column reinforcement details.

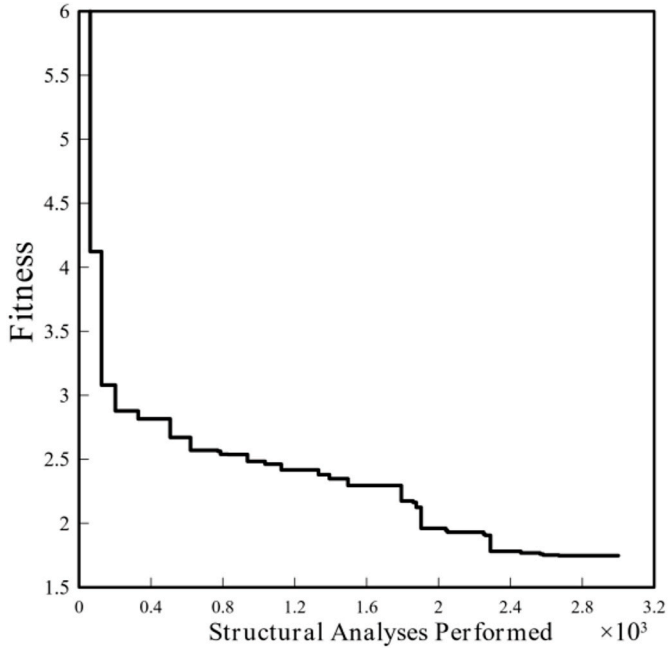


Fig. 14. Convergence histories of the best design for the eight-story structure.

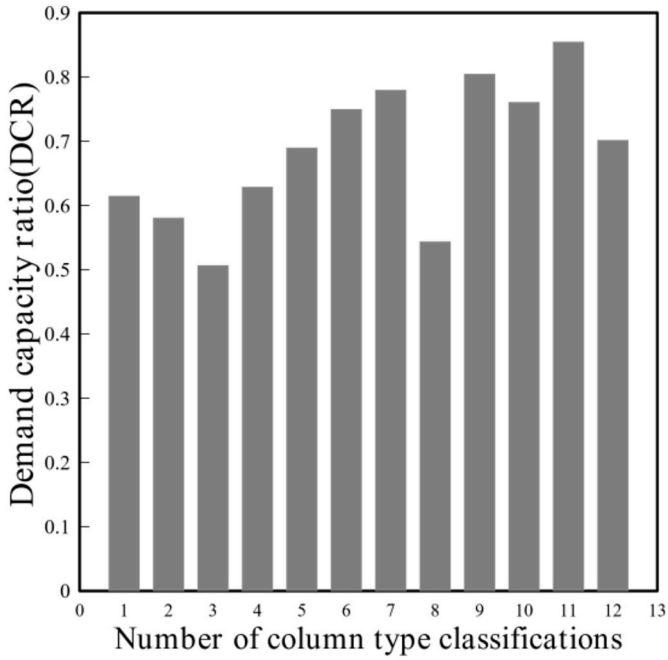


Fig. 15. Maximum DCR of members for each type of classification.

The tabular data presented in Table 6 summarizes the output for one of the optimized structural configurations. It includes key information such as the overall weight of the structure, the dimensions of the individual beam and column groupings, as well as their corresponding reinforcement details. The color-coding of the table columns aligns with the distinct beam and column groups depicted in the 3D structural model shown in Fig. 18. Groups 1–4 represent corner columns, 5–8 are edge columns, and 9–12 are interior columns. For beams, groups 1–4 correspond to exterior beams, while groups 5–8 represent interior beams. The group assignments change at every story. This visual correspondence between the tabular data and the physical model facilitates a more cohesive interpretation of the optimization results.

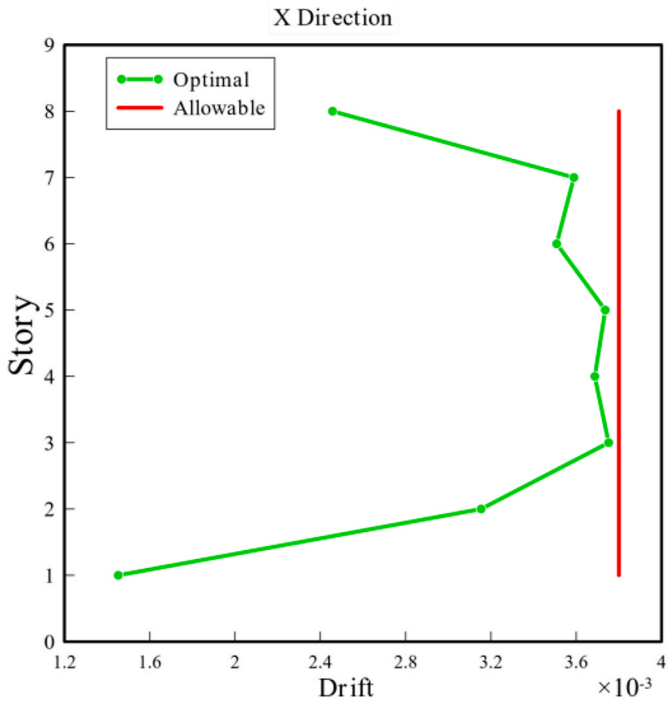


Fig. 16. Drift ratios of the optimum eight-story structure.

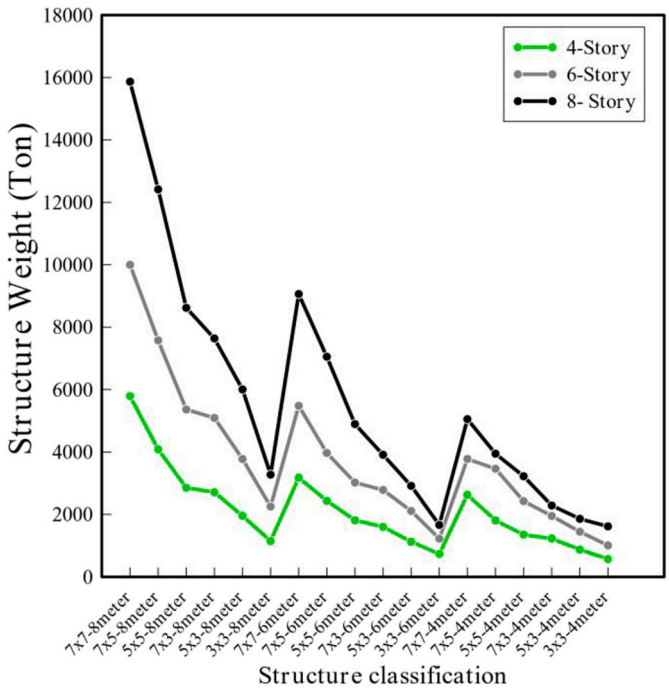


Fig. 17. Best structure weights for all 54 structures.

The machine learning models evaluated in this study include Support Vector Regression (SVR), XGBoost, K-Nearest Neighbors (KNN), Decision Tree, and Random Forest algorithms. To ensure robust model evaluation and minimize overfitting, k-fold cross-validation with $k = 5$ has been employed. This approach divides the dataset into five folds, iteratively using four folds for training and one-fold for testing. Hyperparameter tuning was conducted in conjunction with cross-validation using a grid search strategy. The optimal parameters for each model were determined based on the average performance across all folds, ensuring that the final models

Table 6
Optimization Outputs and Machine Learning Features (A Sample Output for 8-Story SMRF with 7x7 Spans, 8-Meter Width).

Element - Group Name	Dimension	Element - Group Name	Reinforcement	color
Column Section-Group1	C50X50	Column Reinforcement (%)-Group1	3.01	
Column Section-Group2	C50X50	Column Reinforcement (%)-Group2	3.01	
Column Section-Group3	C50X50	Column Reinforcement (%)-Group3	3.01	
Column Section-Group4	C30X30	Column Reinforcement (%)-Group4	1.79	
Column Section-Group5	C80X80	Column Reinforcement (%)-Group5	1.84	
Column Section-Group6	C80X80	Column Reinforcement (%)-Group6	1.84	
Column Section-Group7	C75X75	Column Reinforcement (%)-Group7	1.74	
Column Section-Group8	C40X40	Column Reinforcement (%)-Group8	1.57	
Column Section-Group9	C75X75	Column Reinforcement (%)-Group9	1.74	
Column Section-Group10	C60X60	Column Reinforcement (%)-Group10	2.73	
Column Section-Group11	C50X50	Column Reinforcement (%)-Group11	1.51	
Column Section-Group12	C50X50	Column Reinforcement (%)-Group12	1	
Beam Section-Group1	B50X70	Beam Reinforcement Area (cm^2)-Group1	34.92	
Beam Section-Group2	B50X80	Beam Reinforcement Area (cm^2)-Group2	45.25	
Beam Section-Group3	B50X70	Beam Reinforcement Area (cm^2)-Group3	35.26	
Beam Section-Group4	B30X50	Beam Reinforcement Area (cm^2)-Group4	9.99	
Beam Section-Group5	B30X40	Beam Reinforcement Area (cm^2)-Group5	18.17	
Beam Section-Group6	B40X50	Beam Reinforcement Area (cm^2)-Group6	35.31	
Beam Section-Group7	B30X50	Beam Reinforcement Area (cm^2)-Group7	26.94	
Beam Section-Group8	B30X50	Beam Reinforcement Area (cm^2)-Group8	19.36	
Weight (Ton) = Concrete + Reinforcement		15865.29		

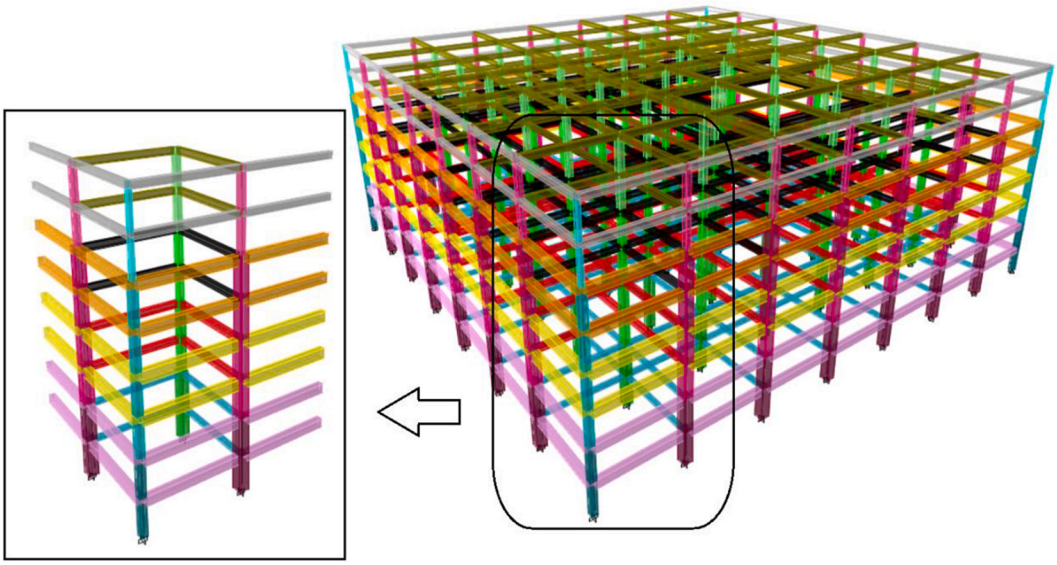


Fig. 18. 3D presentation of the structure showing beam and column groupings, differentiated by color for clarity.

were both accurate and generalizable. The dataset was scaled and normalized to ensure consistency and improve the performance of the machine learning models.

Table 7 summarizes the hyperparameters used for each machine learning model, providing insight into the specific configurations that yielded the best results.

3.6.1. Performance measures for evaluating predictive efficiency

To assess the predictive accuracy of the machine learning models, error rates between actual and predicted values were calculated. As seen in Table 8, to determine the most effective algorithm, four performance metrics were evaluated: R^2 (Coefficient of

Table 7
Hyperparameters for the machine learning models.

Model	Hyperparameter	Range
XGBoost	n_estimators	[100, 200, 500]
	max_depth	[3, 5, 7]
	learning_rate	[0.001, 0.01, 0.1]
	min_child_weight	[1, 3, 5]
Decision Tree	max_depth	[None, 5, 10, 20]
	min_samples_split	[2, 5, 10]
	min_samples_leaf	[1, 2, 5]
	max_features	[None, sqrt, log2]
	ccp_alpha	[0.0, 0.01, 0.1]
SVR	C	[0.1, 1, 10]
	epsilon	[0.1, 0.2, 0.5]
	kernel	[linear, rbf]
Random Forest	n_estimators	[100, 200, 500
	max_depth	[3, 5, 7, None]
	min_samples_split	[2, 3, 5]
KNN	n_neighbors	[3, 5, 7,9]
	weights	[uniform, distance]
	p (Minkowski metric)	[Manhattan, Euclidean]

Determination), MAPE (Mean Absolute Percentage Error), MSE (Mean Squared Error), and MAE (Mean Absolute Error). These metrics provide a comprehensive evaluation of the estimation accuracy and validate the reliability of the predictive results. Detailed formulas for each metric are provided in Table 8.

3.7. Machine learning results

The performance metrics of five machine learning models—XGBoost, Random Forest, Decision Tree, Support Vector Regression (SVR), and K-Nearest Neighbors (KNN)—were evaluated using R^2, Mean Squared Error (MSE), Mean Absolute Error (MEA), and Mean Absolute Percentage Error (MAPE). If significant variation is observed between the performance on the training and test datasets, it indicates potential overfitting. Fig. 19a to e presents the performance measures for both the training and testing datasets. This analysis facilitates the identification of the algorithm that demonstrates the best overall performance [18,41,42].

Given the limited dataset, there is a high likelihood of overfitting, making the gap between training and testing performance a crucial factor in selecting the best model. Among the evaluated models, K-Nearest Neighbors (KNN) achieved perfect performance on the training set, with $R^2 = 1$, MSE = 0, and MAE = 0. However, this exceptional performance strongly suggests overfitting, as indicated by the substantial drop in test performance ($R^2 = 0.69$), resulting in a considerable train-test gap of 0.31. This gap highlights that KNN is likely memorizing the training data rather than learning generalizable patterns, making it unsuitable for this task [20,41].

Among the other models, XGBoost and Random Forest both demonstrated strong performance on the training set, with R^2 values of 0.91 and 0.92, respectively. However, XGBoost outperformed Random Forest in test performance, achieving the highest R^2 of 0.74 while maintaining a smaller gap between training and testing scores (0.17 compared to 0.22 for Random Forest). This suggests that XGBoost generalizes better to unseen data. Moreover, both models exhibited lower mean squared error (MSE) and mean absolute percentage error (MAPE) in comparison to other models, reinforcing their reliability.

Random forest usually has a very good performance when the applied dataset has a small number of features, which is the case for the present dataset [43]. Between Random Forest and XGBoost, XGBoost emerged as the most robust model. While its training R^2 was slightly lower than that of Random Forest, this lower value suggests that XGBoost was less prone to overfitting. The smaller gap between training and testing performance further supports this conclusion, indicating that XGBoost maintained a better balance between bias and variance.

On the other hand, the Decision Tree and Support Vector Regression (SVR) models performed worse than Random Forest and

Table 8
Properties of error metrics.

Name of metric	abbreviation	Meaning	Formulation
Coefficient of Determination	R^2	Variability of the response data around its mean	$1 - \frac{\sum_{i=1}^{n}(A_i - P_i)^2}{\sum_{i=1}^{n}(A_i - \overline{A}_i)^2}$
Mean absolute error	MAE	Averaging of the absolute value of the difference between actual and predicted results for all samples/margin of deviation	$\frac{\sum_{i=1}^{n}\lvert A_i - P_i\rvert}{n}$
Mean squared error	MSE	Average value of squared of deviation number of estimations from actual values/errors for n samples/magnified error	$\frac{\sum_{i=1}^{n}(A_i - P_i)^2}{n}$
Mean Absolute Percentage Error	MAPE	average absolute percentage difference between the actual and predicted values.	$\frac{1}{n}\sum_{i=1}^{n}\left\lvert\frac{A_i - P_i}{A_i}\right\rvert$

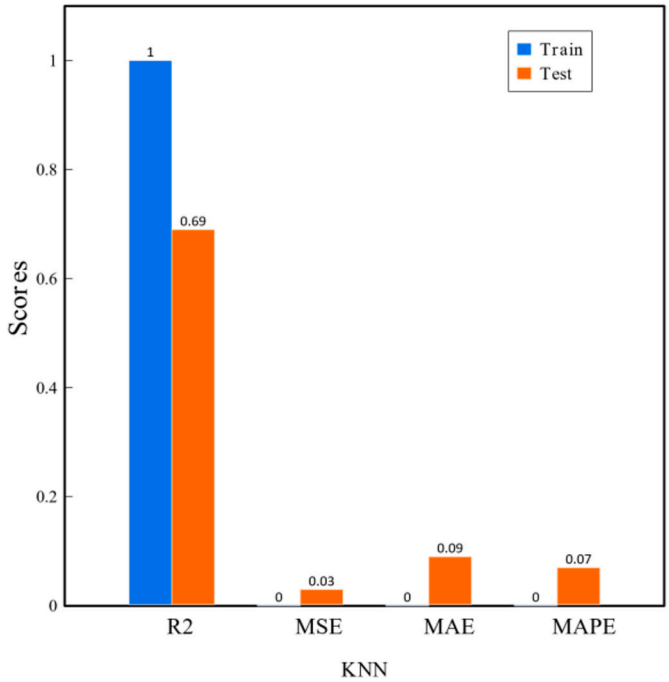


Fig. 19a. Average of performance measures- KNN model.

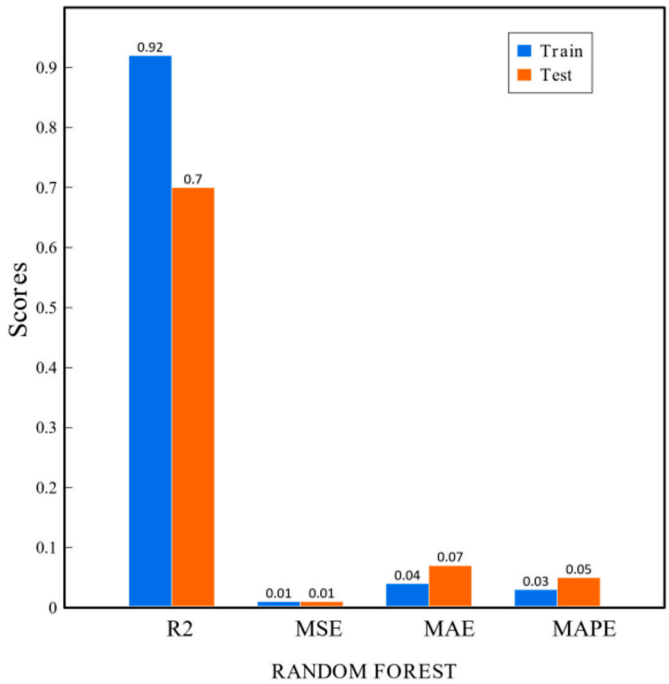


Fig. 19b. Average of performance measures- Random Forest model.

XGBoost. Although SVR achieved a moderate training R^2 of 0.77, its test R^2 dropped to 0.60, showing a larger discrepancy between training and testing performance. Additionally, SVR had higher MAPE values compared to the other models, suggesting that it struggled with generalization. Similarly, the Decision Tree model achieved a training R^2 of 0.85 and a test R^2 of 0.68, indicating overfitting to some extent, though it performed better than SVR in terms of generalizability.

Overall, while KNN exhibited severe overfitting, XGBoost demonstrated the best balance between training and testing performance,

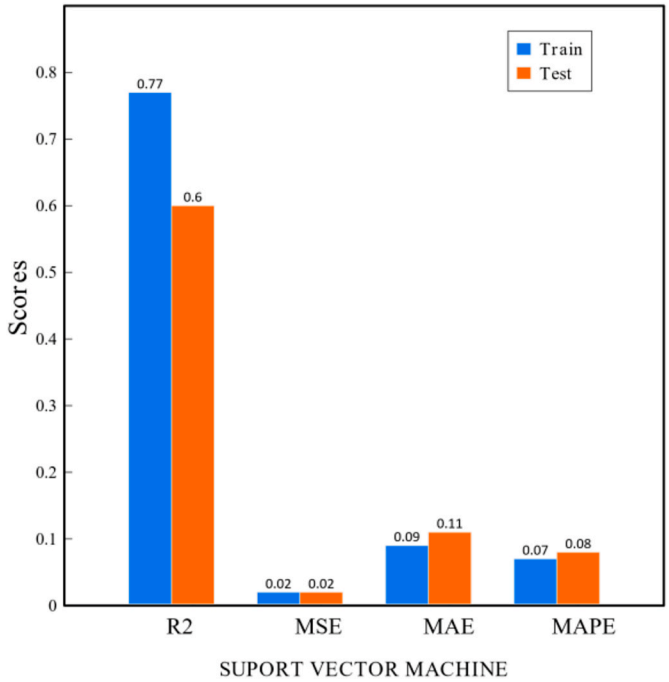


Fig. 19c. Average of performance measures- SVR model.

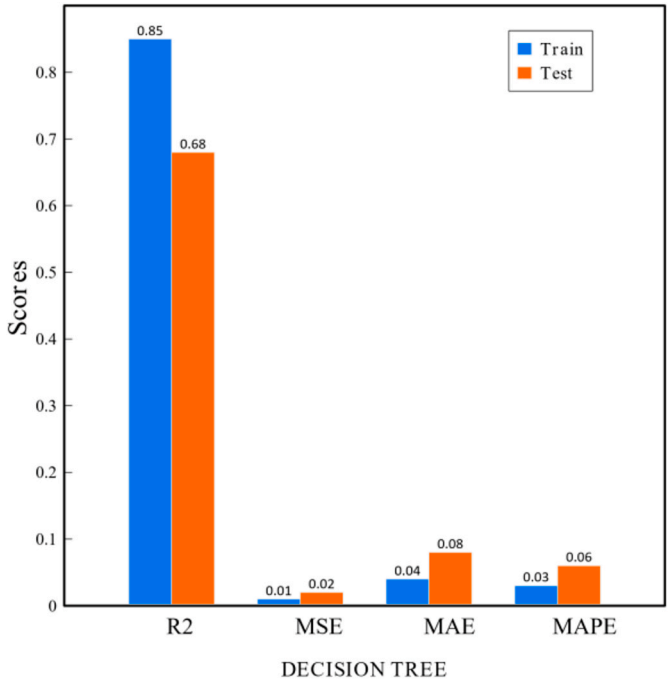


Fig. 19d. Average of performance measures- Decision Tree model.

making it the most suitable model for this dataset. However, XGBoost slow in implementation, due to its use of sequential model training [44].

The feature importance values for input features such as "Story," "L," "N1," and "N2" were calculated using three different machine learning algorithms: XGBoost, Random Forest, and Decision Tree. Here, "Story" refers to the number of stories in the structure, "L" represents the span width, and "N1" and "N2" denote the number of spans in the X and Y directions, respectively. illustrates the feature

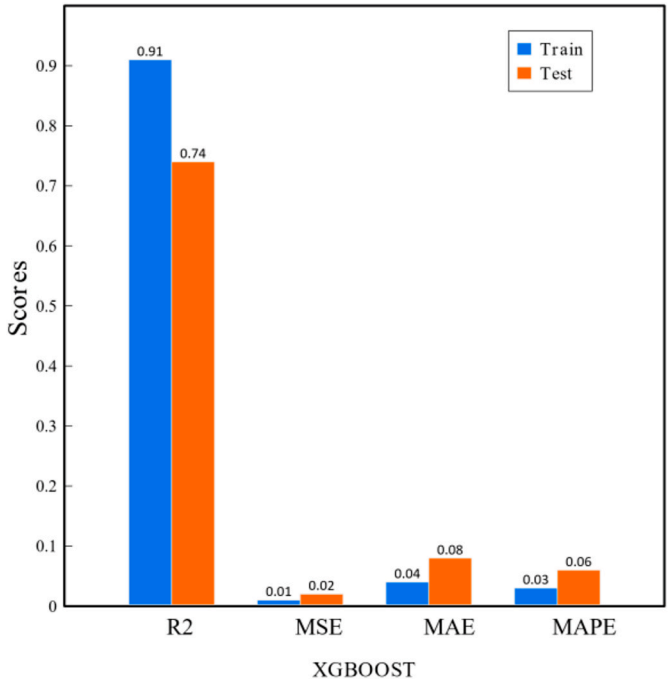


Fig. 19e. Average of performance measures- XGBoost model.

importance plots for each of these three models.

Fig. 20 revealed that "Number of Stories" was the most influential feature in predicting the target variable. This finding aligns with domain knowledge, where the number of stories is expected to have significant predictive power. Increasing the overall height of a structure directly increases earthquake forces, which, in turn, affects the dimensions and reinforcement of beams and columns, ultimately influencing the overall weight of the structure.

The feature importance plot for XGBoost showed a similar trend to Random Forest, with "Number of Stories" dominating the

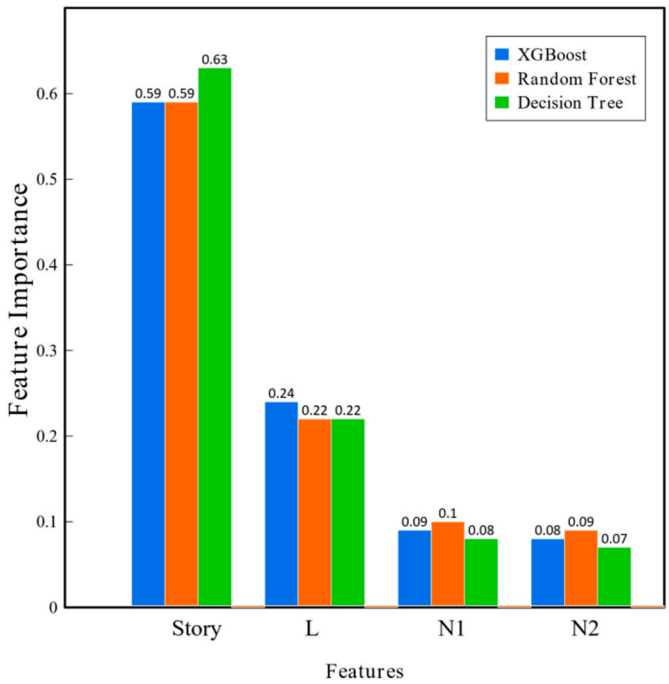


Fig. 20. Feature importance plots for XGBoost, Random Forest, and Decision Tree models.

predictions. However, the overall distribution in the Decision Tree model was less refined, reflecting the simplicity of this algorithm. This may explain its lower generalization performance compared to ensemble methods like XGBoost and Random Forest.

The final objective of this study is to visualize predictive capabilities of models such as XGBoost, Random Forest, SVR, KNN, and Decision Tree. The predictive performance of various machine learning techniques was tested on a dataset that included weight, beam and column dimensions, and rebar percentages. The results presented in Fig. 21 illustrate the relationship between actual and predicted values. Based on the analysis results, it is evident that XGBoost and Random Forest exhibit superior performance compared to other techniques.

The superiority of XGBoost over other models can be attributed to several factors. Firstly, XGBoost incorporates built-in regularization techniques, such as L1 and L2 regularization, which prevent overfitting. This is evident from the smaller gap between training and testing scores compared to Random Forest and Decision Tree. Secondly, XGBoost's gradient boosting mechanism iteratively improves weak learners through a theoretically grounded weighted quantile sketch procedure that enables handling instance weights in approximate tree learning, resulting in better performance and generalization compared to bagging-based methods like Random Forest. Additionally, a parallel and distributed computing capabilities accelerate the learning process, thereby facilitating more efficient model exploration [21,45].

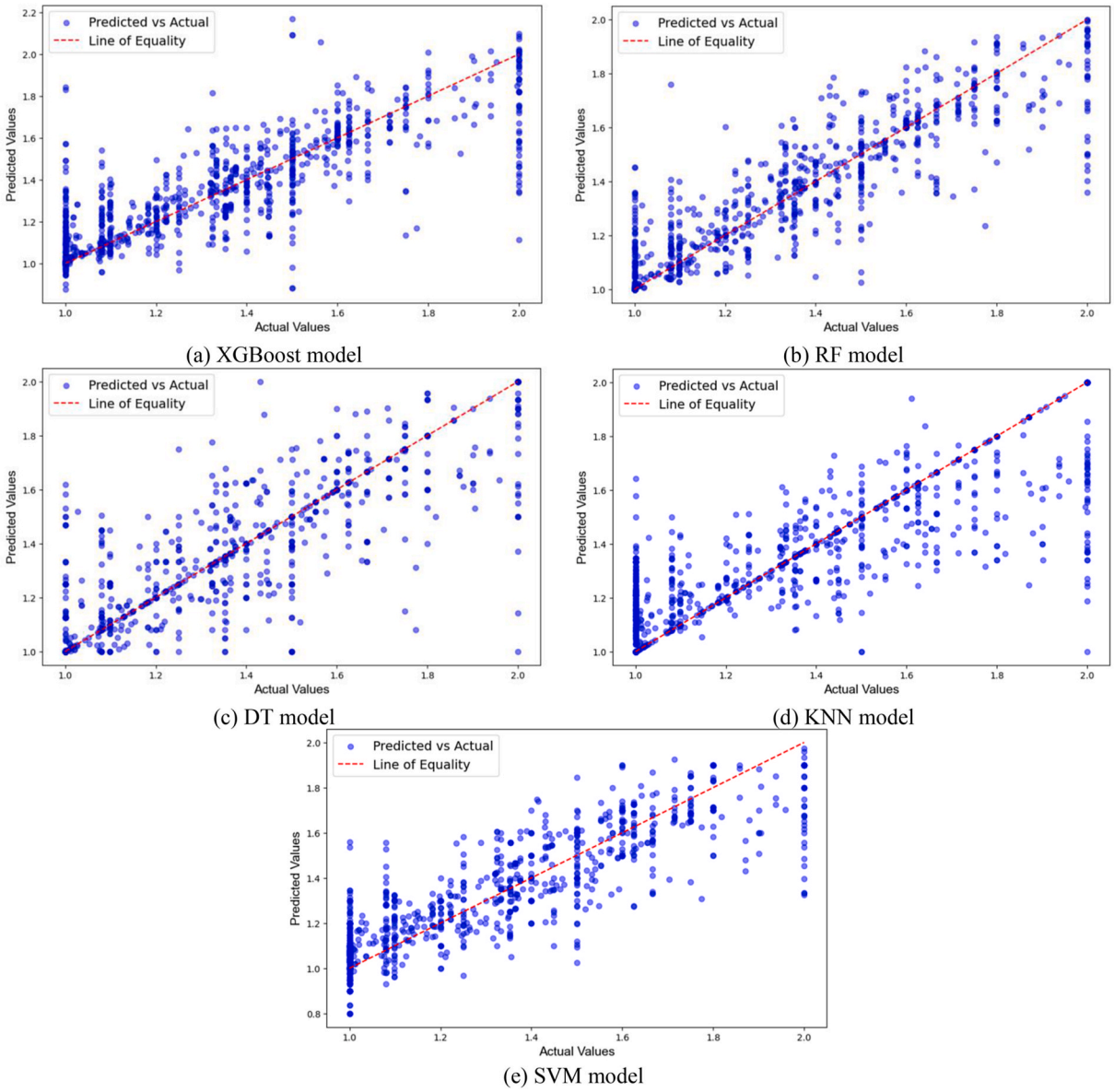


Fig. 21. Comparison between actual and predicted values of various regression techniques: (a) XGBoost model, (b) RF model, (c) DT model, (d) KNN model, (e) SVM model.

Based on the evaluation metrics, XGBoost was determined to be the best-performing model due to its balanced performance, low error rates, and superior generalization capabilities. While KNN, Random Forest and Decision Tree performed well on the training dataset, their overfitting tendencies limited their applicability to unseen data. SVR, while simpler, were less effective compared to tree-based algorithms.

4. Summary and conclusions

This research aimed to optimize the design of concrete structures while adhering to design standards, geometric constraints, and practical construction considerations. The study sought to bridge the gap between research methodologies and real-world design practices to achieve optimal structural performance. Emphasis was placed on critical aspects such as reinforcement detailing, structural weight, and overall cost-efficiency, reflecting the complexities of practical design scenarios. Parameters of the genetic algorithm were refined to manage the diversity of dimensions and weights in analyzed structures. Standardization of weights and constraints ensured a neutral impact of component quantities or group sizes on optimization. This approach allowed independent parameter optimization, leading to robust and adaptable solutions.

Optimization results were utilized to develop machine learning models for designing special concrete moment-resisting frames. The approach, though computationally intensive, provided valuable insights into achieving optimal designs. Structures with varying dimensions (4, 6, and 8 stories) were analyzed to generate a comprehensive dataset. XGBoost demonstrated the best performance (Train $R^2 = 0.91$, Test $R^2 = 0.74$), effectively balancing scalability and capturing nonlinear relationships despite some overfitting. K-Nearest Neighbors (KNN), in other hand, achieved strong training results ($R^2 = 1$) but showed higher sensitivity to overfitting (Test $R^2 = 0.69$). Random Forest performed moderately well, with a large train-test disparity (Train $R^2 = 0.92$, Test $R^2 = 0.7$), indicating limited generalization. Support Vector Regression (SVR struggled significantly on test data ($R^2 = 0.6$). Decision Tree performed well in training ($R^2 = 0.85$) but underperformed in testing ($R^2 = 0.68$). The number of stories was the most influential factor, contributing approximately 60 % to feature importance. This aligns with engineering principles, as taller structures require larger sections and reinforcements due to greater seismic forces. Limited sample size relative to structural features highlighted the need for larger datasets for improved accuracy.

To further enhance the study outcomes, future research should explore structures with varying span lengths, moving beyond the uniform spans analyzed in this work. Comparing the genetic algorithm with alternative optimization methods may identify more efficient solutions for structural optimization tasks. Additionally, incorporating asymmetric structures and employing advanced analysis techniques, such as spectral analysis, could provide deeper insights into complex design scenarios. Expanding the dataset used in the machine learning phase would also likely improve predictive accuracy and model performance. Such advancements would support the development of more robust, efficient, and reliable design tools for concrete structures, enabling broader applications in structural engineering.

CRediT authorship contribution statement

Fateme Heydari: Writing – original draft, Visualization, Validation, Software, Investigation, Formal analysis, Data curation. **Masih Andalib:** Formal analysis, Investigation, Software. **Siamak Epackachi:** Writing – review & editing, Supervision, Resources, Methodology, Investigation, Funding acquisition, Conceptualization. **Reza Rafiee-Dehkharghani:** Writing – review & editing, Supervision, Project administration, Investigation, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] G. Wang, S. Shan, Review of metamodeling techniques in support of engineering design optimization, J. Mechan. Design 129 (Apr. 2007), https://doi.org/10.1115/1.2429697.
[2] B. D'Amico, et al., Machine learning for sustainable structures: a call for data, Structures 19 (Jun. 2019) 1–4, https://doi.org/10.1016/j.istruc.2018.11.013.
[3] C.T. Mueller, Computational Exploration of the Structural Design Space, Ph.D. thesis, Massachusetts Institute of Technology, Cambridge, MA, 2014.
[4] X. Lu, W. Liao, Y. Huang, Z. Zheng, Y. Lin, Automated Structural Design of Shear Wall Residential Buildings Using Generative Adversarial Networks, 2020, https://doi.org/10.21203/rs.3.rs-84863/v1.
[5] G. Mirra, A. Pugnale, Comparison between human-defined and AI-generated design spaces for the optimisation of shell structures, Structures 34 (2021) 2950–2961, https://doi.org/10.1016/j.istruc.2021.09.058.
[6] H. Burton, M. Mieler, Machine Learning Applications in Structural Engineering: Hope, Hype, or Hindrance, 2021.
[7] C. Málaga-Chuquitaype, Machine learning in structural design: an opinionated review, Front. Built Environ. 8 (2022). https://doi.org/10.3389/fbuil.2022.815717.
[8] R. Danhaive, C.T. Mueller, Design subspace learning: structural design space exploration using performance-conditioned generative modeling, Autom. ConStruct. 127 (2021) 103664, https://doi.org/10.1016/j.autcon.2021.103664.

[9] X. Guan, H. Burton, T. Sabol, Python-based computational platform to automate seismic design, nonlinear structural model construction and analysis of steel moment resisting frames, Eng. Struct. 224 (Dec. 2020), https://doi.org/10.1016/j.engstruct.2020.111199.

[10] H. Dehnavipour, M. Mehrabani, A. Fakhriyat, A. Jakubczyk-Gałczyńska, Optimization-based design of 3d reinforced concrete structures, J. Soft Comput. Civil Eng. 3 (3) (Jun. 2019) 95–106, https://doi.org/10.22115/scce.2020.211509.1145.

[11] S. Ulusoy, A.E. Kayabekir, G. Bekdaş, S.M. Nigdeli, Optimum designof reinforced concrete multi-story multi-span frame structures under static loads, Int. J. Eng. Technol. 10 (5) (Oct. 2018) 403–407, https://doi.org/10.7763/IJET.2018.V10.1092.

[12] R. Rempling, A. Mathern, D.T. Ramos, S.L. Fernández, Automatic structural design by a set-based parametric design method, Autom. ConStruct. 108 (Dec. 2019), https://doi.org/10.1016/j.autcon.2019.102936.

[13] J.T. Boscardin, V. Yepes, M. Kripka, Optimization of reinforced concrete building frames with automated grouping of columns, Autom. ConStruct. 104 (Aug. 2019) 331–340, https://doi.org/10.1016/j.autcon.2019.04.024.

[14] M. Mangal, J.C.P. Cheng, Automated optimization of steel reinforcement in RC building frames using building information modeling and hybrid genetic algorithm, Autom. ConStruct. 90 (Jun. 2018) 39–57, https://doi.org/10.1016/j.autcon.2018.01.013.

[15] R. Cazacu, L. Grama, Structural optimization with genetic algorithms and particle swarm optimization, Auo Fmte XXII (1) (2013), https://doi.org/10.15660/AUOFMTE.2013-1.2778 (XII), 2013/1.

[16] M.B.P. Gero, A.B. García, J.del C. Díaz, Design optimization of 3D steel structures: genetic algorithms vs. classical techniques, J. Constr. Steel Res. 62 (12) (Dec. 2006) 1303–1309, https://doi.org/10.1016/j.jcsr.2006.02.005.

[17] R.D. VanLuchene, R.D. Vanluchene, R. Sun, R. Sun, Neural networks in structural engineering, in: Computer-aided Civil and Infrastructure Engineering, 1990, https://doi.org/10.1111/j.1467-8667.1990.tb00377.x.

[18] S. Mangalathu, J.S. Jeon, Classification of failure mode and prediction of shear strength for reinforced concrete beam-column joints using machine learning techniques, Eng. Struct. 160 (Apr. 2018) 85–94, https://doi.org/10.1016/j.engstruct.2018.01.008.

[19] H. Huang, H.V. Burton, Classification of in-plane failure modes for reinforced concrete frames with infills using machine learning, J. Build. Eng. 25 (Sep. 2019), https://doi.org/10.1016/j.jobe.2019.100767.

[20] S. Mangalathu, et al., Classifying earthquake damage to buildings using machine learning, Earthq. Spectra (2020), https://doi.org/10.1177/8755293019878137.

[21] T. Chen, C. Guestrin, XGBoost: a scalable tree boosting system, in: Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Association for Computing Machinery, Aug. 2016, pp. 785–794, https://doi.org/10.1145/2939672.2939785.

[22] S. Bai, M. Li, R. Kong, S. Han, H. Li, L. Qin, Data mining approach to construction productivity prediction for cutter suction dredgers, Autom. ConStruct. 105 (Sep. 2019) 102833, https://doi.org/10.1016/j.autcon.2019.102833.

[23] M.R. Hallowell, A.J.-P. Tixier, AI-based prediction of independent construction safety outcomes from universal attributes, Autom. ConStruct. 118 (Oct. 2020) 103146, https://doi.org/10.1016/j.autcon.2020.103146.

[24] D. Chakraborty, H. Elhegazy, H. Elzarka, L. Gutierrez, A novel construction cost prediction model using hybrid natural and light gradient boosting, Adv. Eng. Inform. 46 (Oct. 2020) 101201, https://doi.org/10.1016/j.aei.2020.101201.

[25] H.H. Elmousalami, Data on field canals improvement projects for cost prediction using artificial intelligence, Data Brief 31 (Aug. 2020) 105688, https://doi.org/10.1016/j.dib.2020.105688.

[26] H. Sun, H.V. Burton, H. Huang, Machine learning applications for building structural design and performance assessment: state-of-the-art review, J. Build. Eng. (2020), https://doi.org/10.1016/j.jobe.2020.101816.

[27] A.W. Charleson, S. Pirie, An investigation of structural engineer-architect collaboration, SESOC 22 (2009) 97–104.

[28] A.S. of C. Engineers, Minimum design loads and associated criteria for buildings and other structures, in: ASCE Standard, American Society of Civil Engineers, 2022 [Online]. Available: https://books.google.com/books?id=lBLNzgEACAAJ.

[29] A.C. Institute, Building Code Requirements for Structural Concrete (ACI 318-19): an ACI Standard ; Commentary on Building Code Requirements for Structural Concrete (ACI 318R-19), American Concrete Institute, ACI Standard, 2019 [Online]. Available: https://books.google.com/books?id=DPJQzQEACAAJ.

[30] S.S. Hosseini, et al., Multi Objective Particle Swarm Optimization (MOPSO) for Size and Shape Optimization of 2D Truss Structures, Periodica Polytechnica-civil Engineering, 2015, https://doi.org/10.3311/ppci.7341.

[31] J.H. Holland, Adaptation in natural and artificial systems: an introductory analysis with applications to biology, in: Control and Artificial Intelligence, MIT Press, Cambridge, MA, USA, 1992.

[32] H. Lingaraj, L. Haldurai, T. Madhubala, R. Rajalakshmi, A study on genetic algorithm and its applications, Int. J. Comput. Sci. Eng. (2016) [Online]. Available: www.ijcseonline.org.

[33] J. Zhong, X. Hu, M. Gu, J. Zhang, Comparison of Performance between Different Selection Strategies on Simple Genetic Algorithms, May 2006, https://doi.org/10.1109/cimca.2005.1631619.

[34] D.E. Goldberg, D.E. Goldberg, K. Deb, K. Deb, A comparative analysis of selection schemes used in genetic algorithms, Foundat. Genetic Algor. (1991), https://doi.org/10.1016/b978-0-08-050684-5.50008-2.

[35] C. Sheppard, Genetic Algorithms with Python, 2019.

[36] T. Bäck, T. Bäck, Evolutionary Algorithms in Theory and Practice: Evolution Strategies, Evolutionary Programming, Genetic Algorithms, 1996 doi: null.

[37] F.-A. Fortin, F.-M. De Rainville, M.-A.G. Gardner, M. Parizeau, C. Gagné, DEAP: evolutionary algorithms made easy, J. Mach. Learn. Res. 13 (1) (Jul. 2012) 2171–2175.

[38] E. Wirsansky, Hands-On Genetic Algorithms with Python: Applying Genetic Algorithms to Solve Real-World Deep Learning and Artificial Intelligence Problems, Packt Publishing Ltd, Birmingham, 2020.

[39] J. Kim, S. Yoo, Software review: DEAP (distributed evolutionary algorithm in Python) library, Springer Science+Business Media 20 (1) (Nov. 2018) 139–142, https://doi.org/10.1007/s10710-018-9341-4.

[40] N. Razali, J. Geraghty, Genetic Algorithm Performance with Different Selection Strategies in Solving TSP, Jan. 2011.

[41] T. Hastie, R. Tibshirani, J. Friedman, The Elements of Statistical Learning, Springer, New York, 2009.

[42] S. Mangalathu, J.-S. Jeon, R. DesRoches, Critical uncertainty parameters influencing seismic performance of bridges using Lasso regression, Wiley 0% Journal Article 47 (3) (2017) 784–801.

[43] J. Kiani, C. Camp, S. Pezeshk, On the application of machine learning techniques to derive seismic fragility curves, Comput. Struct. 218 (Jul. 2019) 108–122, https://doi.org/10.1016/j.compstruc.2019.03.004.

[44] T. Rahman, M.H. Hasan, P. Zheng, Data-driven approach to predict the fundamental period of steel-braced RC frames using stacked generalization machine learning models, Asian J. Civil Eng. (2023), https://doi.org/10.1007/s42107-023-00914-9.

[45] T. Chen, S. Singh, B. Taskar, C. Guestrin, Efficient second-order gradient boosting for conditional random fields, in: G. Lebanon, S.V.N. Vishwanathan (Eds.), Proceedings of the Eighteenth International Conference on Artificial Intelligence and Statistics, Proceedings of Machine Learning Research, vol. 38, PMLR, San Diego, California, USA, May 2015, pp. 147–155 [Online]. Available: https://proceedings.mlr.press/v38/chen15b.html.